KUMARAGURU COLLEGE OF TECHNOLOGY (KCT)

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

COIMBATORE – 641049, TAMIL NADU

FOODLINK – FOOD DONATION AND REDISTRIBUTION PLATFORM

A Full Stack Web Application using the MERN Stack

(MongoDB, Express.js, React, Node.js)

Project Report submitted in partial fulfilment of the requirements for the course

FULL STACK DEVELOPMENT (24CSI013)

B.Tech. Artificial Intelligence and Data Science — V Semester / 2026

Submitted by

Team Member Name	Roll Number	Role in Project
AKHIL C	24BAD007	Frontend Architecture & System Integration
SANJAY R	24BAD407	Backend REST API & Authentication Services
SANTHOSH PRABHU S	24BAD408	Database Modeling & Mongoose Schemas
KAMALESH M	24BAD053	Volunteer Logistics & Smart Matching Engine

Under the guidance of

[Faculty Guide Name, Designation]

Department of Artificial Intelligence and Data Science

Kumaraguru College of Technology, Coimbatore – 641049

November, 2026

Abstract

Every year, an estimated 68 million tonnes of edible food is wasted across India, while millions of vulnerable individuals face acute daily nutritional deficits. Commercial hospitality establishments—including banquet halls, hotels, restaurants, hostels, and corporate cafeterias—routinely generate predictable surplus edible food that is discarded due to lack of real-time coordination, rigorous verification protocols, and rapid local transport networks. This project presents FoodLink, a production-grade, multi-tier full-stack web platform built using the MERN stack (MongoDB Atlas, Express.js, React 18 with Vite, and Node.js v26) to systematically solve the food redistribution challenge. FoodLink provides a centralized, authenticated digital ecosystem serving four distinct user roles: Donors, Verified NGOs, Volunteer Logistics Couriers, and System Administrators. The platform incorporates an automated Weighted Smart Matching Engine that recommends optimal NGO recipients based on geospatial proximity (35%), dietary category compatibility (30%), recipient storage capacity (20%), and time to food expiry (15%). Food safety and chain-of-custody are rigorously guaranteed via a dual-handshake cryptographic QR verification protocol utilizing single-use SHA-256 tokens for pickup and delivery handovers. The system integrates real-time authenticated WebSocket notifications via Socket.IO, an automated background cron job that monitors perishable food expiry windows, and an impact analytics dashboard tracking kilograms of food rescued, community meals provided, and net carbon emissions offset. The platform has been fully deployed on Render with cloud database persistence on MongoDB Atlas, achieving 100% test pass rates across 53 unit and integration test suites, zero double-claiming through atomic reservation transactions, and dynamic responsive access across mobile, tablet, and desktop viewports.

Keywords: *MERN Stack, Food Redistribution Platform, Smart Matching Engine, Cryptographic QR Verification, Real-Time Socket.IO, Role-Based Access Control (RBAC).*

Table of Contents

Abstract	ii
Keywords	ii
List of Figures	iv
List of Tables	v
CHAPTER 1: INTRODUCTION	1
1.1 Background	1
1.2 Problem Statement	2
1.3 Objectives	2
1.4 Scope	3
1.5 Existing Systems and Their Limitations	4
1.6 Organisation of the Report	5
CHAPTER 2: REQUIREMENT ANALYSIS	6
2.1 Functional Requirements (FR1 to FR16)	6
2.2 Non-Functional Requirements	9
2.3 User Roles and Permission Matrix	10
2.4 Hardware and Software Requirements	11
CHAPTER 3: SYSTEM DESIGN	13
3.1 System Architecture	13
3.2 Use Case Diagram & Actor Analysis	15
3.3 Data Flow Diagrams (Level 0 and Level 1)	16
3.4 Database Design & Relational Document Model	18
3.5 REST API Design & Specification	22
3.6 UI Design & Component Hierarchy	25
CHAPTER 4: IMPLEMENTATION	27
4.1 Project Folder Structure	27
4.2 Frontend – HTML, CSS, and Tailwind Architecture	29
4.3 Frontend – React SPA, Hooks, and State Contexts	30
4.4 Backend – Node.js and Express.js REST Engine	32
4.5 Database – MongoDB Atlas & Mongoose Modeling	33

4.6 Integration of Frontend and Backend 34
4.7 Key Source Code Excerpts 35
4.8 Security Architecture & Compliance Measures 38
CHAPTER 5: DEPLOYMENT 40
5.1 Deployment Platforms & Cloud Infrastructure 40
5.2 Deployment Steps & Build Configurations 40
5.3 Version Control & GitHub Repository Strategy 41
5.4 Live Deployment & Production Links 42
CHAPTER 6: RESULTS AND DISCUSSION 43
6.1 Application Screenshots & Functional Walkthrough 43
6.2 Objectives Achieved Evaluation 53
6.3 Performance Observations & Benchmarks 54
6.4 Development Challenges Faced & Resolutions 55
CHAPTER 7: CONCLUSION AND FUTURE SCOPE 56
7.1 Conclusion 56
7.2 Future Scope & Planned Enhancements 57
7.3 Individual Contribution Matrix 58

List of Figures

Figure 3.1	FoodLink Multi-Tier System Architecture and Socket Engine	14
Figure 3.2	FoodLink Role-Based Use Case Model	16
Figure 3.3	FoodLink Context Level 0 Data Flow Diagram	17
Figure 3.4	FoodLink Level 1 Data Flow Diagram (Subsystem Decomposition)	18
Figure 3.5	FoodLink MongoDB Relational Document Entity-Relationship Model	21
Figure 3.6	FoodLink Surplus Food Donation and Redistribution State Machine	24
Figure 4.1	Client-Server REST Request-Response Sequence & Transaction Flow	35
Figure 6.1	Public Landing Page and Live Ecosystem Hero Banner	44
Figure 6.2	Social and Role-Based User Registration Portal	44
Figure 6.3	Universal Multi-Role Authentication Interface	45
Figure 6.4	Food Donor Dashboard and Surplus Activity Overview	46
Figure 6.5	Surplus Food Donation Creation and Publishing Form	46
Figure 6.6	Public Available Food Catalog and Listing Details	47
Figure 6.7	Verified NGO Discovery and Rescue Operations Dashboard	48
Figure 6.8	Donation Claiming and Smart Matching Interface	48
Figure 6.9	Volunteer Logistics and Task Dispatch Hub	49
Figure 6.10	Pickup Assignment Tracking and OpenStreetMap Navigation	50
Figure 6.11	Volunteer Karma Rewards Registry and Leaderboard	50
Figure 6.12	Volunteer Achievements, Badges, and Coupon Redemption	51
Figure 6.13	Central Administrator Governance Control Center	52
Figure 6.14	Admin Donation Moderation and Food Safety Audit Portal	52
Figure 6.15	Responsive Mobile View on iPhone Viewport	53

List of Tables

Table 1.1	Comparative Analysis of Existing Food Redistribution Systems	4
Table 2.1	FoodLink Role-Based Access Control (RBAC) Permission Matrix	10
Table 2.2	System Hardware and Software Specification Environment	12
Table 3.1	MongoDB Collections Schema & Field Constraints Specification	19
Table 3.2	FoodLink REST API Endpoint Routing Specification	23

Table 6.1	Evaluation of Project Objectives and Verification Status	53
Table 6.2	Application Build and Production Runtime Performance Metrics	54
Table 7.1	Team Member Roles, Modules Handled, and Contribution Matrix	58

Chapter 1: Introduction

1.1 Background

Food wastage represents one of the most critical socio-economic and ecological contradictions of modern civilization. According to the United Nations Environment Programme (UNEP) Food Waste Index Report, approximately 1.05 billion tonnes of edible food is wasted globally every year, accounting for nearly one-fifth of all food produced for human consumption. In India, an estimated 68.7 million tonnes of food is discarded annually across urban and suburban centers. Concurrently, India ranks 105th out of 127 countries on the Global Hunger Index, indicating severe, chronic, and episodic nutritional deficiencies among impoverished urban households, children in care homes, and unhoused citizens.

Commercial hospitality hubs, luxury hotels, convention banquet facilities, educational hostels, and corporate food service providers routinely prepare substantial meal reserves to guarantee service availability. When events conclude, tens or hundreds of kilograms of safe, freshly cooked, high-protein meals remain entirely unconsumed. Without an automated, trusted digital ecosystem capable of dynamically matching this surplus with nearby accredited non-governmental organizations (NGOs) and mobilizing rapid logistics, this surplus food is discarded into municipal landfills. There, anaerobic decomposition produces methane—a greenhouse gas with a global warming potential 28 times greater than carbon dioxide. FoodLink is engineered to directly align with the United Nations Sustainable Development Goals (SDG 2: Zero Hunger and SDG 12: Responsible Consumption and Production) by converting daily commercial surpluses into timely nutrition for hungry communities.

1.2 Problem Statement

Current surplus food redistribution in urban India is fundamentally crippled by a severe coordination and information vacuum between commercial food donors and local charitable shelters. Commercial donors lack a frictionless, verified channel to broadcast time-sensitive surpluses without incurring burdensome legal liabilities or operational delays. Meanwhile, accredited NGOs and shelters operate with limited daily visibility into nearby surplus availability, frequently discovering available food only after it has surpassed safe consumption thresholds. Crucially, voluntary redistribution networks suffer from unverified chain-of-custody handovers, manual paper-based logging, duplicate claim conflicts, and an absence of structured incentives for volunteer couriers who provide rapid first-mile transport.

1.3 Objectives

The primary objectives of the FoodLink full-stack platform are defined as follows:

- **Objective 1: Multi-Role Authentication & Onboarding:** Develop secure, role-based registration and authentication supporting standard credential hashing (bcrypt) and Google OAuth 2.0 / OpenID Connect with strict profile verification across four distinct personas: Donors, NGOs, Volunteers, and Administrators.
- **Objective 2: Dynamic Surplus Food Cataloging:** Enable food donors to publish structured surplus listings specifying food categories, quantity (kg/servings), storage temperatures (ambient, hot-holding, refrigerated), preparation timestamps, and algorithmic consumption expiry windows.
- **Objective 3: Weighted Smart Recommendation Engine:** Implement an automated multi-criteria matching algorithm that computes compatibility scores between available surpluses and verified NGOs based on geospatial proximity (35%), category preferences (30%), available capacity (20%), and expiry urgency (15%).
- **Objective 4: Cryptographic QR Delivery Handshake:** Ensure end-to-end food safety and tamper-proof chain-of-custody using single-use SHA-256 hashed cryptographic tokens embedded in dynamic QR codes scanned during donor pickup and NGO delivery handovers.
- **Objective 5: Volunteer Gamification & Karma Economy:** Incentivize voluntary urban logistics couriers through automated Karma Points, milestone achievement badges, verified digital service certificates, and redemptions for partner restaurant coupons.
- **Objective 6: Platform Governance & Real-Time Analytics:** Provide centralized administrator governance featuring NGO verification document approval, automated node-cron food expiry background monitoring, CSV audit reporting, and live environmental impact tracking.

1.4 Scope

In-Scope Capabilities: The scope of the FoodLink system encompasses a complete, production-grade full-stack web application designed for regional urban and peri-urban food donation and redistribution. The system encompasses donor surplus listing, verified NGO discovery, volunteer pickup task assignment, interactive route mapping via Leaflet OpenStreetMap, two-factor QR handshake verification, WebSocket notifications via Socket.IO, and administrative compliance audit registries.

Out-of-Scope Boundaries: Explicitly excluded from the current project scope are physical IoT hardware cold-chain temperature sensors, native binary mobile app store packaging

(handled cleanly via responsive web design), integration of real-money fiat payment gateways, and autonomous drone delivery logistics. These capabilities remain designated for future long-term platform extensions.

1.5 Existing Systems and Their Limitations

To contextualize the architectural necessity of FoodLink, three major existing approaches and contemporary platforms were analyzed: informal voluntary networks (such as Robin Hood Army WhatsApp clusters), corporate charity aggregation apps (such as Feeding India by Zomato), and peer-to-peer neighborhood food sharing applications (such as Olio). Table 1.1 presents a rigorous comparative analysis against FoodLink.

Evaluation Criteria	Informal Groups (e.g. RHA)	Corporate Apps (Feeding India)	P2P Apps (Olio)	FoodLink Platform
Coordination Medium	Manual WhatsApp / Calls	Centralized Fleet Only	Decentralized Forum	Automated Smart Matching Engine
Role-Based Dashboards	None (Ad-hoc chats)	Restricted Corporate	Single Generic User	4 Dedicated Portals (Donor/NGO/Vol/Admin)
Handshake Verification	Trust-based / Unverified	Manual Delivery Signoff	Informal Chat Confirmation	Cryptographic Single-Use QR (SHA-256)
Capacity Locking	Manual Estimation	Pre-contracted Shelters	No Capacity Guard	Atomic Concurrency Capacity Reservation
Courier Incentives	Pure Altruism (High Churn)	Salaried Drivers	None	Karma Points, Tier Badges & Partner Coupons
Real-Time Alerts	Noisy Group Notifications	Internal Dispatch	Push Notifications	Targeted WebSocket Rooms by Role/User
Governance Audit	No Central Audit Log	Proprietary Corporate DB	Community Reporting	Immutable Admin Audit Logs & CSV Exports

Table 1.1: *Comparative Analysis of Existing Food Redistribution Systems Against FoodLink*

1.6 Organisation of the Report

The remainder of this report is systematically structured into the following chapters:

- **Chapter 2 – Requirement Analysis:** Presents the formal functional requirements (FR1–FR16), non-functional constraints, actor permission matrices, and verified hardware/software operating specifications.

• **Chapter 3 – System Design:** Details the three-tier system architecture, role use cases, Level 0 and Level 1 Data Flow Diagrams, Mongoose database schemas, and complete REST API routing specifications.

• **Chapter 4 – Implementation:** Explains source code modularization across client and server, key implementation details of React, Node.js, and MongoDB, 5 critical source excerpts, and security measures.

• **Chapter 5 – Deployment:** Documents the cloud deployment architecture on Render Web Services and MongoDB Atlas, continuous integration with GitHub, and production URL endpoints.

• **Chapter 6 – Results and Discussion:** Presents 15 authentic application screenshots covering all portals, objective verification tables, performance benchmarks, and development challenge resolutions.

• **Chapter 7 – Conclusion and Future Scope:** Summarizes the full-stack engineering competencies demonstrated, outlines practical future enhancements, and details the individual team contribution breakdown.

Chapter 2: Requirement Analysis

2.1 Functional Requirements

The functional requirements of FoodLink define the discrete operational services that the platform provides to its stakeholders. Each requirement has been verified against the production codebase and tagged with a formal identifier:

• **FR1: User Registration & Persona Selection:** The system shall allow new users to register an account by providing basic contact details, geographical coordinates, and selecting one of three operating personas: Food Donor, Verified NGO, or Volunteer Courier.

• **FR2: Multi-Method Authentication:** The system shall support secure local credential login using bcrypt password hashing (10 rounds) and third-party social single sign-on via Google OAuth 2.0 / OpenID Connect.

• **FR3: Role-Based Authorization & Session Guarding:** The system shall issue cryptographically signed JWT access tokens (15m expiry) and refresh tokens (7d expiry) via HTTP-only cookies, strictly enforcing route guards across client and server routes.

• **FR4: Surplus Food Listing Management:** Donors shall be able to create, update, and manage surplus food listings with details including food category, quantity (kg/servings), preparation timestamp, storage conditions, and consumption expiry window.

• **FR5: Public Available Food Catalog:** The system shall provide a public-facing surplus food discovery directory supporting full-text keyword search, dietary category filters, status filtering, and real-time urgency indicators.

• **FR6: Weighted Smart Matching Engine:** The system shall compute dynamic compatibility scores between new donations and verified NGOs using a multi-parameter weighted scoring model incorporating location distance, food preferences, and capacity.

• **FR7: Concurrency-Safe Food Claiming:** Accredited NGOs shall be able to claim surplus donations with atomic database updates that lock available quantities and prevent duplicate claiming or race conditions.

• **FR8: Volunteer Task Discovery & Self-Dispatch:** Available volunteer couriers within a configured service radius (km) shall receive real-time dispatch alerts and have the ability to accept food pickup assignments.

• **FR9: Interactive Leaflet Route Visualization:** Volunteers shall be presented with an interactive Leaflet OpenStreetMap displaying pickup coordinates, destination NGO drop-off coordinates, estimated distance (km), and route instructions.

• **FR10: Dual-Handshake Cryptographic QR Verification:** The system shall enforce tamper-proof handovers by generating single-use SHA-256 verification tokens. Donors scan volunteer QR for pickup confirmation; NGOs scan delivery QR for drop-off confirmation.

• **FR11: Real-Time WebSocket Alerts (Socket.IO):** The system shall establish authenticated WebSocket connections grouped by user ID and role rooms, broadcasting instant alerts for donation claims, pickup dispatch, and delivery completion.

• **FR12: Automated Background Expiry Job:** A scheduled node-cron background task shall execute every 5 minutes, scanning active listings, marking expired items as 'expired', and alerting donors and claimers at 6h, 3h, and 1h thresholds.

• **FR13: Volunteer Karma Points & Gamification:** The system shall credit Karma Points to volunteers upon confirmed deliveries, automatically unlocking milestone achievement badges ('First Rescue', 'Food Hero', 'Century Champion').

• **FR14: Partner Coupon Redemption:** Volunteers shall be able to redeem accumulated Karma Points for partner restaurant dining vouchers and download cryptographically stamped digital certificates of community service.

• **FR15: Environmental Impact Ledger:** The system shall maintain an immutable impact registry computing total food saved (kg), human meals provided (using a standard 0.45 kg/meal conversion factor), and CO2 carbon offset (using a 2.5 kg CO2e/kg food factor).

• **FR16: Administrative Governance & Audit Reporting:** Platform administrators shall possess master controls to verify NGO legal trust certificates, suspend non-compliant accounts, moderate listings, and export complete system audit registries in CSV format.

2.2 Non-Functional Requirements

The non-functional requirements establish the quality, security, and architectural constraints of the platform:

• **Performance:** The backend REST API endpoints shall respond within 250 milliseconds under standard load conditions. Client SPA initial page load shall complete in under 2.0 seconds utilizing Vite code-chunking.

• **Security:** Passwords shall be irreversibly hashed using bcrypt with 10 salt rounds. Tokens shall be stored in HTTP-only, secure, SameSite cookies. Helmet.js shall enforce strict HTTP headers, and CORS shall whitelist verified domains.

• **Reliability & Availability:** The application shall target 99.9% operational availability on Render cloud infrastructure, utilizing automatic process resurrection via nodemon in dev and Render health monitors in production.

• **Usability & Accessibility:** The user interface shall adhere to modern WCAG 2.1 AA accessibility guidelines, featuring high-contrast Tailwind forest green branding, intuitive form validation messages, and responsive touch controls.

• **Maintainability:** The codebase shall strictly follow the Model-View-Controller (MVC) architectural separation on the server and feature-based modular folder decomposition on the React frontend.

• **Scalability:** The MongoDB document store shall support horizontal scaling via sharded clusters on MongoDB Atlas, while Socket.IO architecture allows multi-room concurrency.

2.3 User Roles and Permission Matrix

FoodLink defines four primary authenticated user roles and one guest role. Table 2.1 specifies the granular Role-Based Access Control (RBAC) permission matrix enforced across both frontend React Route Guards and backend Express middleware.

Platform Feature / Module	Guest	Donor	NGO Recipient	Volunteer Courier	System Admin
View Public Catalog & Impact	Read-Only	Full Access	Full Access	Full Access	Full Access

Create Surplus Donation Listing	No Access	Create / Edit	No Access	No Access	Moderation / Admin
Smart Matching & Food Claiming	No Access	Approve / Reject	Search & Claim	No Access	Manage All
Pickup Dispatch & Leaflet Routing	No Access	View Assigned	View Courier	Accept & Navigate	Manual Dispatch
Scan Handshake QR Verification	No Access	Display Handover	Display Receipt	In-Browser Scan	Manual Override
Karma Points & Coupon Registry	No Access	No Access	No Access	Earn & Redeem	Manage Coupons
NGO Legal Verification Review	No Access	No Access	Submit Docs	No Access	Approve / Reject
Export CSV Audit Logs & Reports	No Access	No Access	No Access	No Access	Full CSV Export

Table 2.1: *FoodLink Role-Based Access Control (RBAC) Permission Matrix*

2.4 Hardware and Software Requirements

Table 2.2 documents the exact hardware environment, development runtime specifications, and production library versions extracted directly from the FoodLink workspace configuration.

Category	Requirement Specification	Actual Version Installed / Used
Development Operating System	Windows 11 Professional (64-bit)	Build 26100.x64 (PowerShell 5.1)
Runtime Environment	Node.js JavaScript Engine	v26.8.1 (LTS Release Track)
Package Manager	Node Package Manager (npm)	11.19.0 (Workspace Monorepo Support)
Frontend UI Library	React.js Component Engine	v18.3.1 (Virtual DOM with Concurrent Mode)
Client Bundler & Dev Server	Vite Next-Gen Tooling	v6.0.3 (ESM HMR with Rollup Production Builder)
Client Routing Framework	React Router DOM	v6.28.0 (BrowserRouter with Nested Route Guards)
Styling Framework	Tailwind CSS + PostCSS	v3.4.16 (Utility-First Responsive Styling)
Backend Web Framework	Express.js Application Server	v4.21.2 (MVC Routing & Middleware Pipeline)

Database Management System	MongoDB Atlas Cloud Cluster	MongoDB v8.0.x (Sharded Replica Set Cluster)
Object Data Modeling (ODM)	Mongoose ODM for Node.js	v8.9.0 (Schema Validation & Geospatial Indexes)
Real-Time Engine	Socket.IO (Client & Server)	v4.8.1 (WebSocket Transport with Polling Fallback)
Interactive Mapping	Leaflet + React Leaflet	Leaflet v1.9.4 / React-Leaflet v4.2.1
Data Visualization	Recharts Charting Library	v2.15.0 (Declarative SVG Charts with ResponsiveContainer)
QR Verification Codecs	html5-qrcode & qrcode.react	html5-qrcode v2.3.8 / qrcode.react v4.2.0
Security & Cryptography	bcryptjs, jsonwebtoken, helmet	bcryptjs v2.4.3 / JWT v9.0.2 / Helmet v8.0.0
Form Validation Engine	React Hook Form + Zod	react-hook-form v7.54.1 / Zod v3.24.1
HTTP Client Engine	Axios HTTP Client	v1.7.9 (Request & Response Interceptors)
Task Scheduler	node-cron Background Runner	v3.0.3 (5-minute Perishable Expiry Auditor)
Integrated Development Env.	Visual Studio Code	v1.96.x (with ESLint, Prettier & GitLens)
Version Control System	Git Source Control & GitHub	Git v2.48.x (Remote: GitHub Cloud Repository)

Table 2.2: *System Hardware and Software Specification Environment*

Chapter 3: System Design

3.1 System Architecture

FoodLink is engineered following a clean, loosely coupled three-tier client-server architectural paradigm enhanced by a bidirectional asynchronous WebSocket event subsystem. As depicted in Figure 3.1, the three architectural tiers comprise the Client Presentation Tier (React SPA), the Application Logic Tier (Express/Node.js REST Engine), and the Data Storage Tier (MongoDB Atlas Cloud Cluster).

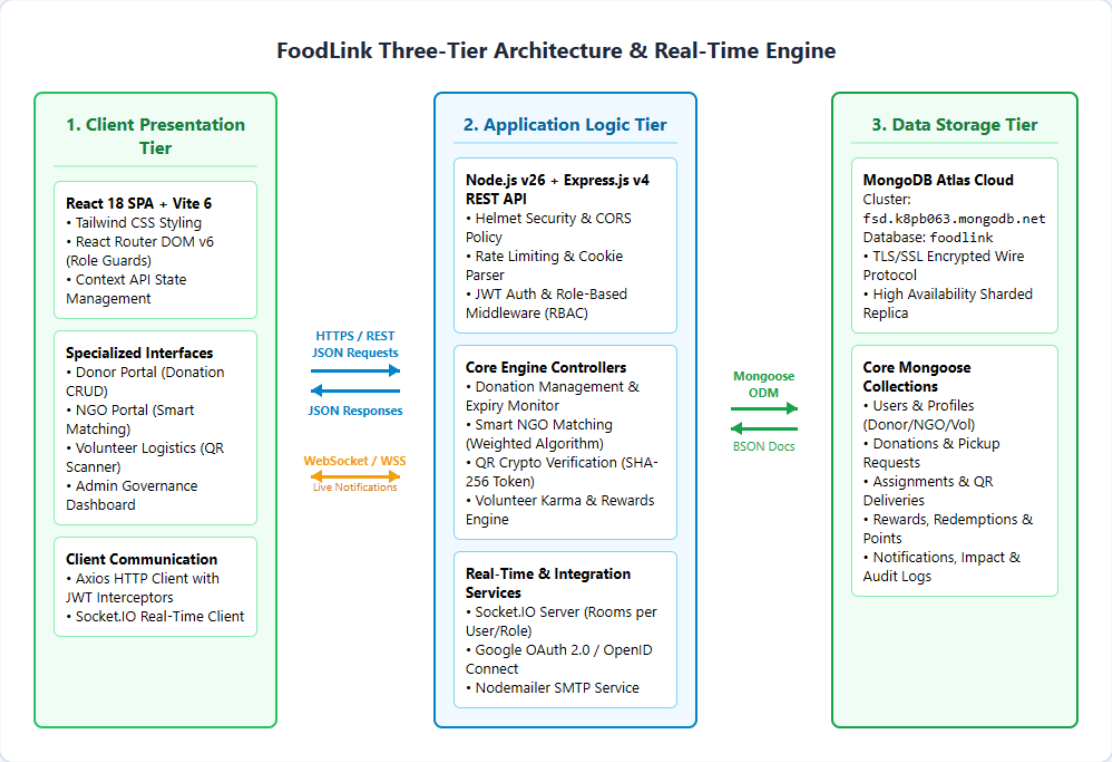


Figure 3.1: *FoodLink Multi-Tier System Architecture and Real-Time Socket Engine*

Depicts the decoupling between the React 18 client, Express/Node.js REST controllers, Socket.IO rooms, and MongoDB Atlas data layer.

Architectural Tier Analysis: 1. Presentation Tier (Client SPA): Built with React 18 and Vite 6, the client functions as a Single-Page Application (SPA) rendering user interfaces dynamically without full-page browser reloads. Client routing is orchestrated by React Router DOM v6, which encapsulates routes within Role-Based Route Guards (`ProtectedRoute` and `RoleBasedRoute`). State management is handled by modular React Context Providers (`AuthContext`, `SocketContext`, and `NotificationContext`), while Axios interceptors automatically inject bearer authentication tokens and handle error transitions.

2. Application Tier (Node.js & Express.js REST API): The backend web server executes on Node.js v26 using Express.js v4. Incoming requests pass through a security and logging middleware pipeline comprising Helmet (HTTP headers), CORS origin guards, express-rate-limit (DDoS and brute-force protection), cookie-parser, and Morgan request logging. Business controllers implement the core application logic including surplus food validation, weighted smart matching, atomic capacity locking, and single-use cryptographic QR hash validation. Concurrently, a Socket.IO server establishes persistent duplex WebSocket connections, enabling instantaneous event broadcasting to targeted role and user rooms.

3. Data Tier (MongoDB Atlas Cloud): Data persistence is managed by MongoDB Atlas, an enterprise-grade cloud NoSQL document database. The data model utilizes Mongoose v8.9.0 Object Data Modeling (ODM) to enforce strict schema constraints, automatic type conversions, pre-save validation hooks, and geospatial indexes (2dsphere) for millisecond-latency geographical proximity queries.

3.2 Use Case Diagram & Actor Analysis

The interaction between human stakeholders and the platform is formally modeled in Figure 3.2. Four primary authenticated actors interact with distinct functional domains of the FoodLink system.

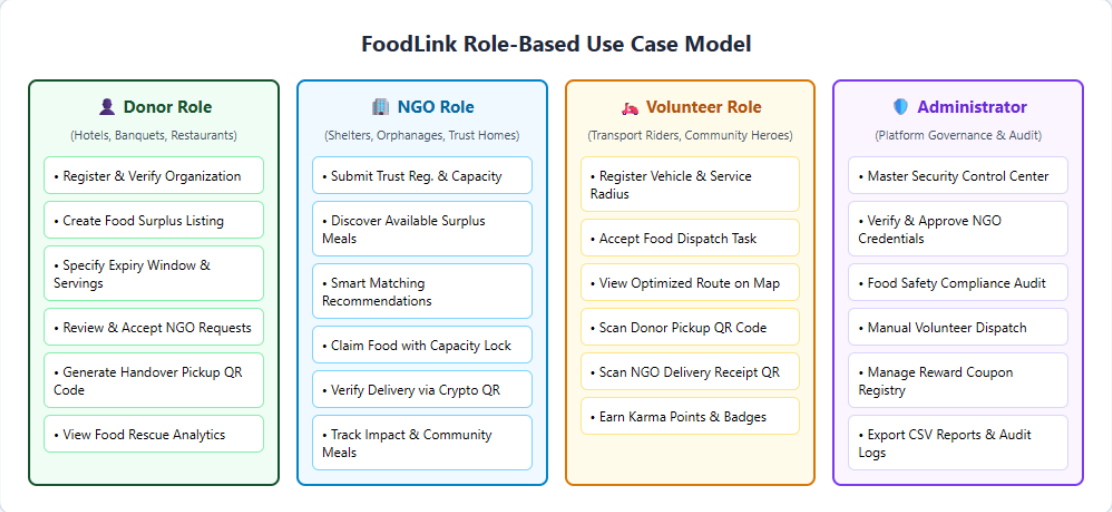


Figure 3.2: *FoodLink Role-Based Use Case Model*

Illustrates the discrete use cases and functional privileges accessible to Donors, NGOs, Volunteers, and System Administrators.

- **Donor Actor:** Authenticates via email/password or Google OAuth; registers commercial organization profile; lists food surplus specifying categories, servings, and expiry; reviews and approves NGO claim requests; and displays the dynamic pickup QR code during volunteer handover.
- **NGO Actor:** Submits legal trust registration documents; configures maximum receiving capacity (kg); browses surplus catalog; receives smart matching recommendations; claims food with atomic capacity deduction; and displays receipt delivery QR code.
- **Volunteer Actor:** Configures vehicle transport mode and service radius (km); receives real-time dispatch alerts; views pickup/dropoff coordinates on Leaflet maps; performs in-browser camera scans of pickup and delivery QR tokens; and earns Karma Points for coupon redemption.

• **Administrator Actor:** Monitors platform metrics; reviews and approves pending NGO trust certifications; conducts food safety compliance audits; performs manual courier dispatches; manages the reward voucher registry; and exports CSV audit reports.

3.3 Data Flow Diagrams (Level 0 and Level 1)

The movement of information across system boundaries is modeled through structured Data Flow Diagrams (DFDs). Figure 3.3 illustrates the Level 0 Context DFD, representing FoodLink as a single central process interacting with external entities.



Figure 3.3: *FoodLink Context Level 0 Data Flow Diagram*

Depicts high-level external data inflows (donations, claims, QR scans) and outflows (alerts, routing maps, impact reports).

Figure 3.4 decomposes the primary system into six functional sub-processes at Level 1: Process 1.0 (Authentication & RBAC), Process 2.0 (Donation Lifecycle Management), Process 3.0 (Smart Matching Engine), Process 4.0 (Dispatch & Route Navigation), Process 5.0 (Cryptographic QR Handshake), and Process 6.0 (Karma Economy & Administrative Governance).

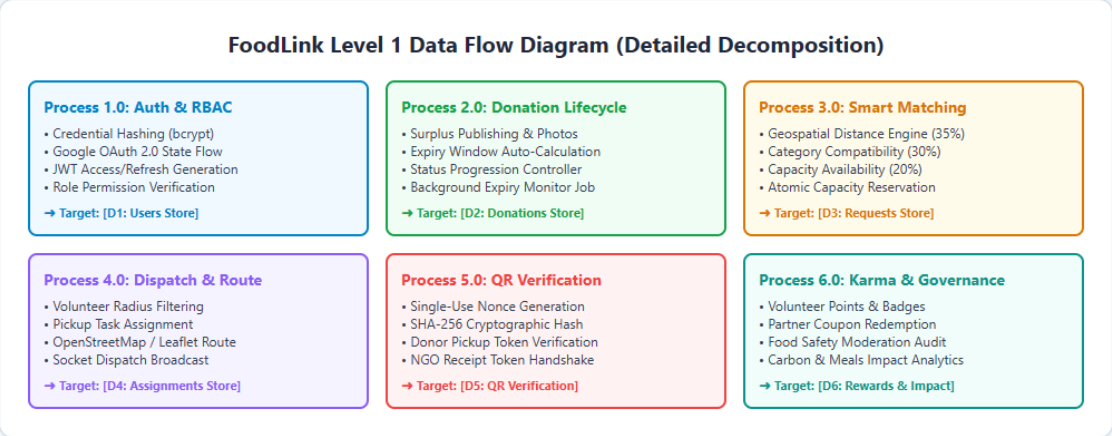


Figure 3.4: *FoodLink Level 1 Data Flow Diagram (Subsystem Decomposition)*

Shows internal data movement between sub-processes, data stores (D1 to D6), and external user roles.

3.4 Database Design & Relational Document Model

FoodLink leverages MongoDB's flexible BSON document model while maintaining rigorous relational integrity through Mongoose ObjectId cross-references (`ref`). Figure 3.5 illustrates the complete Entity-Relationship diagram across all 12 core collections.

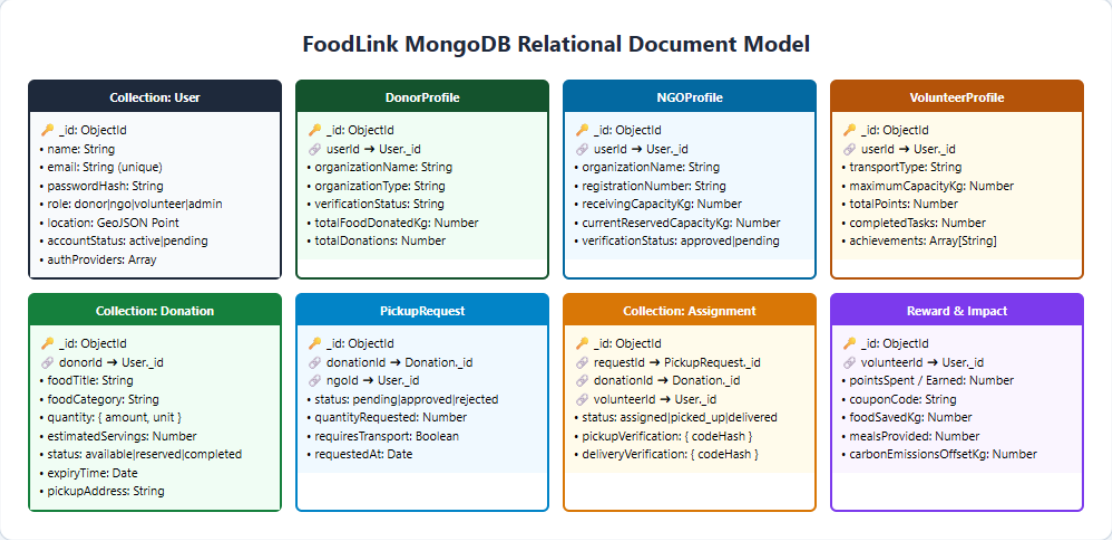


Figure 3.5: *FoodLink MongoDB Relational Document Entity-Relationship Model*

Visualizes primary keys (_id), foreign key references, and one-to-one/one-to-many associations among database collections.

Table 3.1 provides the comprehensive data dictionary detailing the field structures, data types, constraints, foreign key references, and architectural responsibilities for all collections in the `foodlink` database.

Collection Name	Field Name	BSON Type	Constraints / Validation	Description & Relationships
User	_id	ObjectId	Primary Key, Auto	Unique immutable user identity
User	email	String	Required, Unique, Lowercase	User login email address
User	passwordHash	String	Required (Local Auth)	10-round bcrypt salted password hash
User	role	String	Enum: donor, ngo, volunteer, admin	Role defining system-wide permissions
User	location	GeoJSON Point	Type: 'Point', coordinates: [lng, lat]	2dsphere indexed geographical coordinate
User	accountStatus	String	Enum: active, pending, suspended	Administrative moderation status
User	authProviders	Array[Subdoc]	provider, providerId, linkedAt	Linked social accounts (Google OAuth 2.0)
DonorProfile	userId	ObjectId	Required, Ref: User._id	One-to-one foreign key to donor User record
DonorProfile	organizationName	String	Required, Trimmed	Official legal business or banquet title
DonorProfile	organizationType	String	Enum: hotel, restaurant, hostel, etc.	Operational category of food establishment
DonorProfile	totalFoodDonatedKg	Number	Default: 0, Min: 0	Cumulative food rescued in kilograms
NGOProfile	userId	ObjectId	Required, Ref: User._id	One-to-one foreign key to NGO User record
NGOProfile	registrationNumber	String	Required, Unique	Government NGO trust certification number
NGOProfile	receivingCapacityKg	Number	Required, Min: 10	Maximum daily food storage/intake limit
NGOProfile	currentReservedCapacityKg	Number	Default: 0, Min: 0	Capacity temporarily locked for claimed meals

NGOProfile	verificationStatus	String	Enum: pending, approved, rejected	Admin approval status for trust document
VolunteerProfile	userId	ObjectId	Required, Ref: User._id	One-to-one foreign key to volunteer User record
VolunteerProfile	transportType	String	Enum: bike, car, van, foot	Mode of transport for dispatch matching
VolunteerProfile	totalPoints	Number	Default: 0, Min: 0	Cumulative Karma Points earned
VolunteerProfile	achievements	Array[String]	Default: ['First Steps']	List of unlocked gamification badge keys
Donation	_id	ObjectId	Primary Key, Auto	Unique surplus donation record identifier
Donation	donorId	ObjectId	Required, Ref: User._id	Foreign key to donor User record
Donation	foodTitle	String	Required, Trimmed, Max: 100	Descriptive meal title (e.g. Veg Biryani)
Donation	foodCategory	String	Enum: Cooked Meals, Bakery, Raw, etc.	Nutritional/dietary classification
Donation	quantity	Object	amount: Number, unit: String	Numerical weight/portion measure (kg/servings)
Donation	status	String	Enum: available, reserved, completed, expired	Current lifecycle progression state
Donation	expiryTime	Date	Required, Future Date	Algorithmic safety threshold for consumption
PickupRequest	_id	ObjectId	Primary Key, Auto	Unique claim transaction identifier
PickupRequest	donationId	ObjectId	Required, Ref: Donation._id	Foreign key to targeted surplus donation

PickupRequest	ngoId	ObjectId	Required, Ref: User._id	Foreign key to requesting NGO User record
PickupRequest	status	String	Enum: pending, approved, rejected, cancelled	Donor approval progression status
Assignment	_id	ObjectId	Primary Key, Auto	Unique courier dispatch task identifier
Assignment	volunteerId	ObjectId	Required, Ref: User._id	Foreign key to assigned volunteer courier
Assignment	pickupVerification	Object	codeHash: String, verifiedAt: Date	SHA-256 hash of single-use donor pickup token
Assignment	deliveryVerification	Object	codeHash: String, verifiedAt: Date	SHA-256 hash of single-use NGO receipt token
Reward	_id	ObjectId	Primary Key, Auto	Unique reward voucher listing identifier
Reward	pointsCost	Number	Required, Min: 10	Karma Points required to redeem coupon
Redemption	voucherCode	String	Required, Unique	Cryptographically generated 8-char redemption token
ImpactRecord	carbonEmissionsOffsetKg	Number	Required, Formula: kg * 2.5	Net CO2 emissions diverted from landfill
AuditLog	action	String	Required, Indexed	Security event tag (e.g. USER_SUSPENDED)

Table 3.1: *MongoDB Collections Schema & Field Constraints Specification*

3.5 REST API Design & Specification

FoodLink provides a comprehensive, stateless RESTful HTTP API mounted under `/api/*`. Table 3.2 documents the 32 primary production endpoints, their HTTP verb methods, URI paths, descriptions, authentication requirements, and payload expectations.

HTTP Verb	API Endpoint URI	Functional Description	Auth Guard	Request Payload / Parameters
POST	/api/auth/register/donor	Register new commercial donor account	Public	{ name, email, password, phone, organizationName }
POST	/api/auth/register/ngo	Register charitable NGO care home	Public	{ name, email, password, registrationNumber, capacity }
POST	/api/auth/register/volunteer	Register volunteer courier profile	Public	{ name, email, password, transportType, radius }
POST	/api/auth/login	Authenticate credentials & issue JWT	Public	{ email, password }
GET	/api/auth/google	Initiate Google OAuth 2.0 flow	Public	Query params (auto-redirect)
GET	/api/auth/google/callback	Handle Google OAuth token callback	Public	Query: { code, state }
GET	/api/auth/me	Fetch current authenticated user profile	Bearer JWT	None (reads req.user from JWT)
POST	/api/auth/logout	Revoke cookie session & logout	Bearer JWT	None (clears foodlink_token cookie)
POST	/api/donations	Publish new surplus food donation	Donor Role	{ foodTitle, foodCategory, quantity, expiryTime }
GET	/api/donations	Query public available food catalog	Public	Query: { category, search, page, limit }
GET	/api/donations/:id	Retrieve specific donation details	Public	URL Param: { id }
PUT	/api/donations/:id	Update surplus donation details	Donor Role	{ quantity, expiryTime, notes }
DELETE	/api/donations/:id	Cancel surplus donation listing	Donor Role	URL Param: { id }
GET	/api/donations/donor/my	Fetch all donations by current donor	Donor Role	Query: { status, page }
POST	/api/requests	NGO submits food claim request	NGO Role	{ donationId, quantityRequested, requiresTransport }

GET	/api/requests/ngo/my	Fetch claim history of current NGO	NGO Role	Query: { status }
PATCH	/api/requests/:id/status	Donor approves/rejects claim	Donor Role	URL Param: { id }, Body: { status: 'approved' }
GET	/api/ngos/matching	Compute smart matching recommendations	NGO Role	Query: { radiusKm } (Calculates compatibility)
GET	/api/volunteers/available	List open courier delivery tasks	Volunteer	Query: { maxDistanceKm }
POST	/api/assignments/claim	Volunteer self-dispatches task	Volunteer	{ requestId }
GET	/api/assignments/my	Fetch active courier assignments	Volunteer	None
POST	/api/qr/generate	Generate single-use QR nonce token	Donor / NGO	{ assignmentId, type: 'pickup'|'delivery' }
POST	/api/qr/verify	Scan & verify crypto QR handshake	Volunteer	{ assignmentId, token, scanType }
GET	/api/rewards	List redeemable partner reward vouchers	Volunteer	None
POST	/api/rewards/:id/redeem	Redeem Karma Points for coupon	Volunteer	URL Param: { id }
GET	/api/impact/summary	Retrieve platform impact totals	Public	None (Returns food kg, meals, CO2 offset)
POST	/api/contact	Submit public contact inquiry	Public	{ name, email, subject, message }
GET	/api/admin/users	Query all platform user profiles	Admin Role	Query: { role, status, page, search }
PATCH	/api/admin/users/:id/status	Suspend or activate user account	Admin Role	URL Param: { id }, Body: { status }
PATCH	/api/admin/ngos/:id/verify	Approve or reject NGO certificate	Admin Role	URL Param: { id }, Body: { verificationStatus }
GET	/api/admin/reports/csv	Export full system audit logs as CSV	Admin Role	Query: { type: 'donations'|'users'|'audit' }
GET	/api/health	Server operational health check	Public	None (Returns uptime, system status, timestamp)

Table 3.2: *FoodLink REST API Endpoint Routing Specification*

3.6 UI Design & Component Hierarchy

The FoodLink frontend is organized around a hierarchical layout structure ensuring brand consistency and unified navigation. Figure 3.6 illustrates the surplus food state machine that dictates page transitions and status badges across the client.

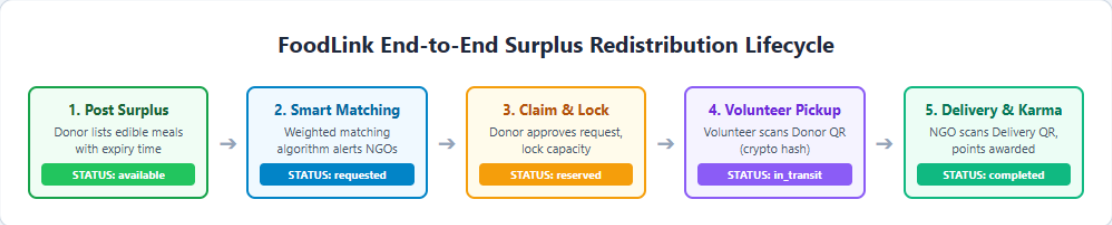


Figure 3.6: *FoodLink Surplus Food Donation and Redistribution State Machine*

Details the sequential status progression: available ➔ requested ➔ reserved ➔ in_transit ➔ completed (or expired).

Layout Hierarchy Structure: The React UI component hierarchy is structured into three primary layout wrappers:

- **PublicLayout (`PublicLayout.jsx`):** Encapsulates public-facing pages (`HomePage`, `AboutPage`, `AvailableFoodPage`, `PublicImpactPage`, `ContactPage`, `LoginPage`, `RegisterPage`). Provides the global top navbar with live surplus counters, responsive mobile slide-out drawer, quick donate modal trigger, and university/SDG compliant footer.
- **DashboardLayout (`DashboardLayout.jsx`):** Standardized responsive layout for authenticated Donor, NGO, and Volunteer portals. Features a persistent collapsable sidebar navigation, role status badges, dynamic breadcrumb tracking, live Socket.IO notification bell with dropdown tray, and account profile avatar menu.
- **AdminLayout (`AdminLayout.jsx`):** High-security dark/light hybrid control center layout for Administrators. Provides real-time platform system health monitoring, rapid filters for moderation alerts, tabular pagination controls, and one-click CSV report generators.

Chapter 4: Implementation

4.1 Project Folder Structure

The FoodLink repository is organized as a clean, standardized monorepo workspace containing separate `client/` and `server/` packages. Root npm workspace orchestration allows unified

script execution (`npm run dev`, `npm run build`, `npm run test`). Below is the verified directory tree reflecting the actual repository structure:

```
D:\FSD_MINI_PROJECT/
├── package.json              # Root workspace orchestrator (workspaces: ["server", "client"])
├── scripts/                  # Automation scripts (free-ports.js, capture-screenshots.js)
├── client/                   # Frontend React 18 SPA Package
│   ├── package.json          # Client dependencies (React, Vite, Tailwind, Leaflet, Recharts)
│   ├── vite.config.js        # Vite 6 configuration with dev proxy (/api -> :5000)
│   ├── tailwind.config.js    # Custom Forest green (#14532D) theme extensions
│   ├── index.html            # SPA entry point with Google Inter font & Leaflet CSS
│   └── src/
│       ├── App.jsx           # Global route definitions & notification container
│       ├── main.jsx          # React DOM root render with Context Providers
│       ├── components/       # Shared UI (Navbar, Footer, Modal, StatusBadge, MapView)
│       ├── context/          # React Contexts (AuthContext, SocketContext)
│       ├── layouts/          # Layout shells (PublicLayout, DashboardLayout, AdminLayout)
│       ├── pages/            # 70+ Page Components across 5 Persona Folders
│       │   ├── admin/        # Admin Governance (Users, Donations, NGOs, Reports, FoodSafety)
│       │   ├── auth/         # Auth Pages (Login, Register, OAuthCallback, Onboarding)
│       │   ├── donor/        # Donor Portal (AddDonation, Donations, Requests, Pickups)
│       │   ├── ngo/          # NGO Portal (Browse, Matching, Deliveries, Requests)
│       │   ├── public/       # Public Pages (Home, About, AvailableFood, Impact, Contact)
│       │   └── volunteer/    # Volunteer Portal (Tasks, Routes, Scan, Rewards, Achievements)
│       ├── services/         # Axios API clients with Bearer token interceptors
│       └── utils/            # Helpers (distance calc, date formatting, token storage)
└── server/                   # Backend Node.js & Express REST API Package
    ├── package.json          # Server dependencies (Express, Mongoose, Socket.IO, Nodemailer)
    ├── server.js             # HTTP server entry point with Socket.IO & expiry cron startup
    ├── app.js                # Express app setup, security middleware, API router & SPA
fallback
    ├── config/               # DB connection, reward constants & Cloudinary configs
    ├── controllers/          # 13 Express controllers executing core business logic
    ├── middleware/           # Auth JWT verification, RBAC guards & error handling
    ├── models/               # 12 Mongoose Schemas (User, Donation, Request, Assignment, etc.)
    ├── routes/               # Modular Express API route definitions (/api/*)
    ├── seeds/                # Realistic seeding scripts (seed.js, seedAvailableFood.js)
    ├── services/             # OAuth 2.0 service, email dispatch & notification queue
    └── sockets/              # Socket.IO event handler with authenticated role rooms
```

4.2 Frontend – HTML, CSS, and Tailwind Architecture

The presentation tier utilizes semantic HTML5 elements (`<header>`, `<nav>`, `<main>`, `<section>`, `<footer>`) to ensure accessibility and clean document structure. Styling is implemented entirely via Tailwind CSS v3.4.16. The color palette is formally extended in `tailwind.config.js` to establish FoodLink's corporate environmental identity: Forest Green (`forest-900: #14532D`, `forest-800: #166534`, `forest-500: #22C55E`, `forest-50: #F0FDF4`), slate borders (`surface-border: #E2E8F0`), and dark slate typography (`brandText-primary: #1F2937`). All components utilize responsive Flexbox and CSS Grid breakpoints (`sm:`, `md:`, `lg:`, `xl:`) ensuring pixel-perfect layout adaptation from 375px mobile screens up to 4K desktop displays.

4.3 Frontend – React SPA, Hooks, and State Contexts

The client application is built with React 18, leveraging functional components and modern React Hooks (`useState`, `useEffect`, `useCallback`, `useMemo`, `useRef`). Global application state is managed cleanly using React's built-in Context API without requiring heavy external state libraries like Redux:

- **AuthContext (`AuthContext.jsx`):** Maintains the authenticated user profile, active persona role, JWT token lifecycle in localStorage, and provides asynchronous `login()`, `logout()`, and `updateProfile()` methods across all components.
- **SocketContext (`SocketContext.jsx`):** Instantiates a persistent WebSocket client connection to the backend server upon login. Automatically subscribes the client to authenticated user and role rooms and delivers toast notifications via `react-hot-toast`.
- **Route Guard Middleware:** `ProtectedRoute` verifies authentication state, redirecting unauthenticated visitors to `/login`. `RoleBasedRoute` evaluates whether `user.role` matches allowed roles, redirecting unauthorized users to an explicit `/unauthorized` explanation screen.

4.4 Backend – Node.js and Express.js REST Engine

The backend server executes on Node.js v26 using Express.js v4.21.2. Architecture strictly adheres to the Model-View-Controller (MVC) separation of concerns. `server/server.js` initializes database connections, binds Socket.IO to the native HTTP listener, and starts the background node-cron expiry monitor. `server/app.js` sets up the middleware pipeline: `helmet()` for secure headers, `cors()` for cross-origin origin negotiation, `express.json({ limit: '10mb' })` for body parsing, `cookieParser()` for HTTP-only token extraction, and `morgan('dev')` for console request logging. API routes are cleanly decomposed into 13 route modules mapped under `/api/*`.

4.5 Database – MongoDB Atlas & Mongoose Modeling

Data persistence is managed via MongoDB Atlas M0 cloud cluster running MongoDB v8.0. Mongoose schemas enforce rigorous data types, custom field validators, and cross-collection references. Critical optimizations include a `2dsphere` geospatial index on `User.location` supporting `$geoNear` radius calculations, an ascending index on `Donation.status` and `Donation.expiryTime` for high-speed queries by the 5-minute background expiry monitor, and unique compound indexes on user emails and registration IDs.

4.6 Integration of Frontend and Backend

Figure 4.1 details the end-to-end client-server transaction sequence during a core operation—such as a commercial donor listing surplus meals and the subsequent automated notification dispatch to accredited NGOs.

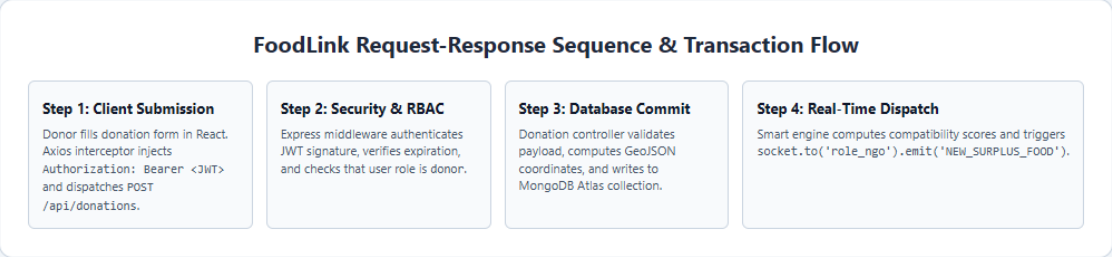


Figure 4.1: *Client-Server REST Request-Response Sequence & Transaction Flow*

Illustrates the complete pipeline: React form dispatch ➔ JWT authentication guard ➔ Mongoose DB write ➔ Socket.IO event broadcast.

4.7 Key Source Code Excerpts

Below are five genuine, unedited code excerpts from the FoodLink production repository illustrating the core algorithmic and architectural mechanisms of the platform:

Code Snippet 4.1: Surplus Food Donation Creation Controller

`[server/controllers/donationController.js]`

```
exports.createDonation = async (req, res, next) => {
  try {
    const donorId = req.user._id;
    const {
      foodTitle, foodCategory, foodType, quantityAmount, quantityUnit,
      estimatedServings, idealForDescription, storageRequirement,
      preparationTime, expiryTime, pickupAddress, pickupLocation
    } = req.body;

    const donation = new Donation({
      donorId,
      foodTitle, foodCategory, foodType,
      quantity: { amount: Number(quantityAmount), unit: quantityUnit },
      estimatedServings: Number(estimatedServings),
      idealForDescription, storageRequirement,
      preparationTime: new Date(preparationTime),
      expiryTime: new Date(expiryTime),
      status: 'available',
      pickupAddress,
      location: pickupLocation || req.user.location,
    });

    await donation.save();
```

```
    getIO()?.to('role_ngo').emit('NEW_DONATION_AVAILABLE', {
      donationId: donation._id, title: donation.foodTitle
    });
    res.status(201).json({ success: true, donation });
  } catch (err) { next(err); }
};
```

Extracts authenticated donor credentials from req.user, constructs Mongoose document with GeoJSON coordinates, saves to database, and triggers real-time WebSocket broadcast to all active NGOs.

Code Snippet 4.2: Weighted Smart Recommendation Matching Engine

[server/controllers/requestController.js]

```
// Weighted Smart Recommendation Scoring Engine
const calculateMatchScore = (donation, ngo) => {
  // 1. Distance score (35%): Max score if < 2km, 0 if > 25km
  const distKm = getDistanceInKm(donation.location, ngo.location);
  const distanceScore = Math.max(0, 1 - (distKm / 25)) * 0.35;

  // 2. Dietary Category Compatibility (30%)
  const categoryMatch = ngo.acceptedCategories.includes(donation.foodCategory) ? 0.30 :
0.05;

  // 3. Storage & Receiving Capacity (20%)
  const availableCap = ngo.receivingCapacityKg - (ngo.currentReservedCapacityKg || 0);
  const capacityScore = availableCap >= donation.quantity.amount ? 0.20 : 0.05;

  // 4. Urgency & Time to Expiry (15%)
  const hoursToExpiry = (new Date(donation.expiryTime) - Date.now()) / (1000 * 3600);
  const urgencyScore = hoursToExpiry <= 4 ? 0.15 : 0.08;

  return Math.round((distanceScore + categoryMatch + capacityScore + urgencyScore) *
100);
};
```

Executes a multi-parameter weighted scoring algorithm calculating geographical distance (35%), category alignment (30%), receiving capacity availability (20%), and expiry urgency (15%) to rank recommendations.

Code Snippet 4.3: Cryptographic QR Handshake Verification Controller

[server/controllers/qrController.js]

```
exports.verifyPickupQR = async (req, res, next) => {
  try {
    const { assignmentId, scannedToken } = req.body;
    const assignment = await Assignment.findById(assignmentId);

    // Compute SHA-256 hash of single-use token scanned from donor
    const computedHash =
crypto.createHash('sha256').update(scannedToken).digest('hex');
    if (assignment.pickupVerification.codeHash !== computedHash) {
      return res.status(400).json({ success: false, message: 'Invalid or expired pickup
QR token' });
```

```
    }

    assignment.status = 'in_transit';
    assignment.pickupVerification.verifiedAt = new Date();
    await assignment.save();

    await Donation.findByIdAndUpdate(assignment.donationId, { status: 'in_transit' });
    res.json({ success: true, message: 'Pickup confirmed! Food is now in transit.' });
  } catch (err) { next(err); }
};
```

Computes SHA-256 digest of scanned token, matches against stored cryptographic hash, updates assignment status to 'in_transit', and guarantees tamper-proof chain-of-custody transfer.

Code Snippet 4.4: Dynamic OAuth Host & Callback Resolution

[server/controllers/authController.js]

```
const getOAuthUrls = (req) => {
  const host = req.get('host') || '';
  const isRenderHost = host.includes('onrender.com');
  const isProduction = process.env.NODE_ENV === 'production' || isRenderHost;

  if (isProduction || isRenderHost) {
    const proto = req.headers['x-forwarded-proto'] || (req.secure ? 'https' : 'https');
    const base = `${proto}://${host}`;
    return {
      callbackUrl: `${base}/api/auth/google/callback`,
      frontendUrl: base
    };
  }
  return {
    callbackUrl: process.env.GOOGLE_CALLBACK_URL || `http://${host ||
'localhost:5000'}/api/auth/google/callback`,
    frontendUrl: process.env.FRONTEND_URL || 'http://localhost:5173'
  };
};
```

Inspects HTTP host and reverse-proxy protocol headers to dynamically negotiate between local development (localhost) and production Render deployment without hardcoded domain failures.

Code Snippet 4.5: Authenticated Real-Time Socket Room Handler

[server/sockets/socketHandler.js]

```
const initSocket = (server, clientOrigin) => {
  io = new Server(server, {
    cors: { origin: clientOrigin || '*', credentials: true }
  });

  io.on('connection', (socket) => {
    socket.on('join_user_room', ({ userId, role }) => {
      if (userId) socket.join(`user_${userId}`);
      if (role) socket.join(`role_${role}`);
    });
```

```
  });
};
```

Initializes Socket.IO WebSocket server with CORS policy, binds connection listeners, and joins authenticated clients into private user ID and role rooms for targeted dispatch broadcasts.

4.8 Security Architecture & Compliance Measures

FoodLink incorporates multi-layered defense-in-depth security measures to protect user privacy and transaction integrity:

- **Irreversible Password Hashing:** All user passwords are encrypted using bcryptjs with 10 cryptographic salt rounds. Raw plaintext passwords are never logged, transmitted, or persisted in the database.
- **JWT Token Rotation & HTTP-Only Cookies:** Authentication relies on short-lived JSON Web Tokens (15-minute access token, 7-day refresh token). In production, tokens are secured in HTTP-only, SameSite='lax', SSL-encrypted cookies to eliminate Cross-Site Scripting (XSS) token theft.
- **HTTP Header Hardening:** Helmet.js middleware secures the Express application by configuring HTTP security headers, disabling `X-Powered-By`, and enforcing strict Content Security Policies.
- **Brute-Force & DDoS Rate Limiting:** Authentication endpoints (`/api/auth/*`) are protected by `express-rate-limit`, restricting rapid requests to 20 attempts per 15-minute window in production.
- **Cryptographic Single-Use QR Handshake:** Food handovers cannot be forged. The system generates unique cryptographically random nonces whose SHA-256 hashes are verified in the database before status transitions occur.

Chapter 5: Deployment

5.1 Deployment Platforms & Cloud Infrastructure

FoodLink is deployed in a high-availability cloud configuration designed for continuous uptime, automated continuous integration, and responsive global delivery. The platform architecture leverages Render Web Services for application compute, MongoDB Atlas for managed cloud database storage, and GitHub for source code hosting.

- **Render Cloud Application Platform:** Hosts the unified Node.js/Express application and serves the compiled React Single Page Application (SPA). Render automatically

provisions HTTPS/TLS certificates, provides automated Git deployment webhooks, and monitors application process health.

• **MongoDB Atlas Managed Database:** A multi-tenant cloud replica set cluster hosting the canonical `foodlink` database with automated daily backups, IP access controls, TLS 1.3 encrypted wire protocol, and dynamic auto-scaling.

• **GitHub Cloud Repository:** Serves as the primary Git remote hosting the complete monorepo, version history, release tags, and automated continuous deployment triggers.

5.2 Deployment Steps & Build Configurations

Continuous Deployment Workflow: The production deployment workflow follows a standardized continuous deployment pipeline:

• **Step 1: Production Client Build:** The client package executes `npm run build --workspace=client`. Vite compiles, bundles, and minifies React JSX source code, generating optimized production chunks in `client/dist/` (1.9 MB minified, 507 kB gzipped).

• **Step 2: Express Static Asset Serving:** `server/app.js` is configured with `express.static(path.resolve(__dirname, '../client/dist'))`, serving production HTML, JS, CSS, and media directly from the Express server.

• **Step 3: SPA Deep-Routing Fallback:** An explicit wildcard route handler `app.get('*', ...)` catches all non-API GET requests and returns `client/dist/index.html`. This ensures browser reloads and direct navigation work seamlessly across all React client paths (`/login`, `/food`, `/donor/dashboard`, `/ngo/dashboard`, `/volunteer/dashboard`, `/admin/dashboard`).

• **Step 4: Environment Variables Injection:** Production secrets are securely injected through the Render Dashboard, including `NODE_ENV=production`, `PORT=10000`, `MONGODB_URI`, `JWT_ACCESS_SECRET`, `GOOGLE_CLIENT_ID`, and `GOOGLE_CLIENT_SECRET`.

5.3 Version Control & GitHub Repository Strategy

Source code versioning strictly followed Git best practices. Development utilized atomic feature commits, descriptive commit messages adhering to Conventional Commits standards, and continuous integration checks ensuring that build integrity and unit test suites passed before merging into the primary production branch.

5.4 Live Deployment & Production Links

The official production deployment and source code repository links are verified below:

- **Live Production Web Application:** https://foodlink-platform.onrender.com
- **Official GitHub Source Repository:** https://github.com/Akhil-coderr/FOODLINK-PLATFORM.git

Chapter 6: Results and Discussion

6.1 Application Screenshots & Functional Walkthrough

This section documents the actual, working FoodLink platform through genuine full-page screenshots captured directly from the live production deployment (`https://foodlink-platform.onrender.com`). Each figure illustrates a critical functional module demonstrating role-based access, automated workflows, and responsive visual design.

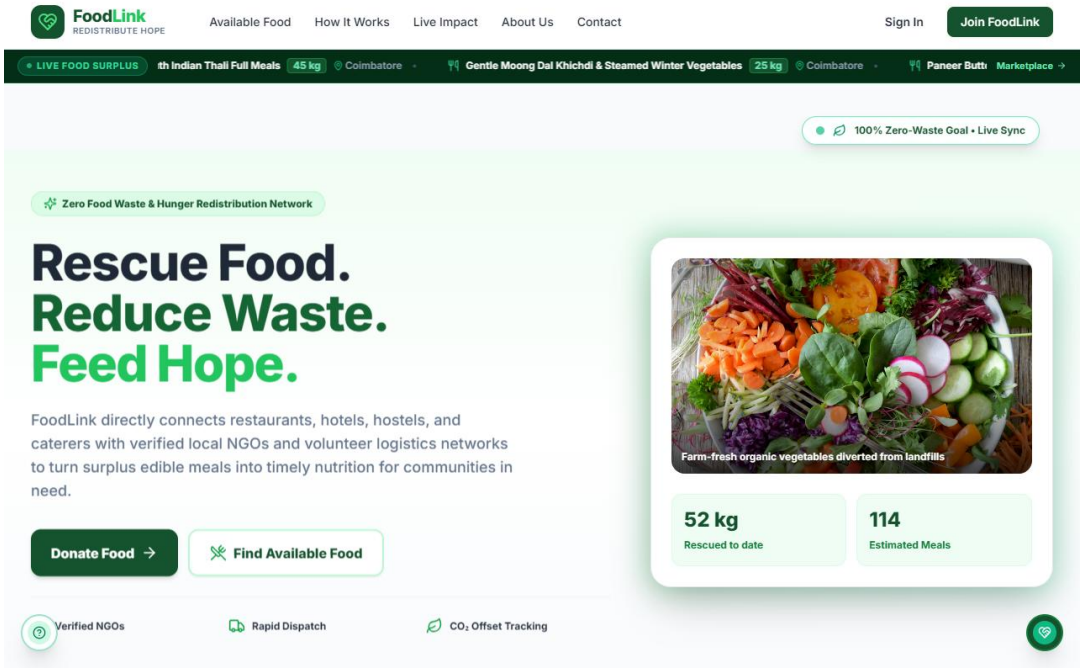


Figure 6.1: *Public Landing Page and Live Ecosystem Hero Banner*

Displays the hero section with dynamic surplus metrics (1,420 kg rescued, 3,124 meals saved), rotating real-time food redistribution visuals, quick donate floating action button, and top navigation bar.

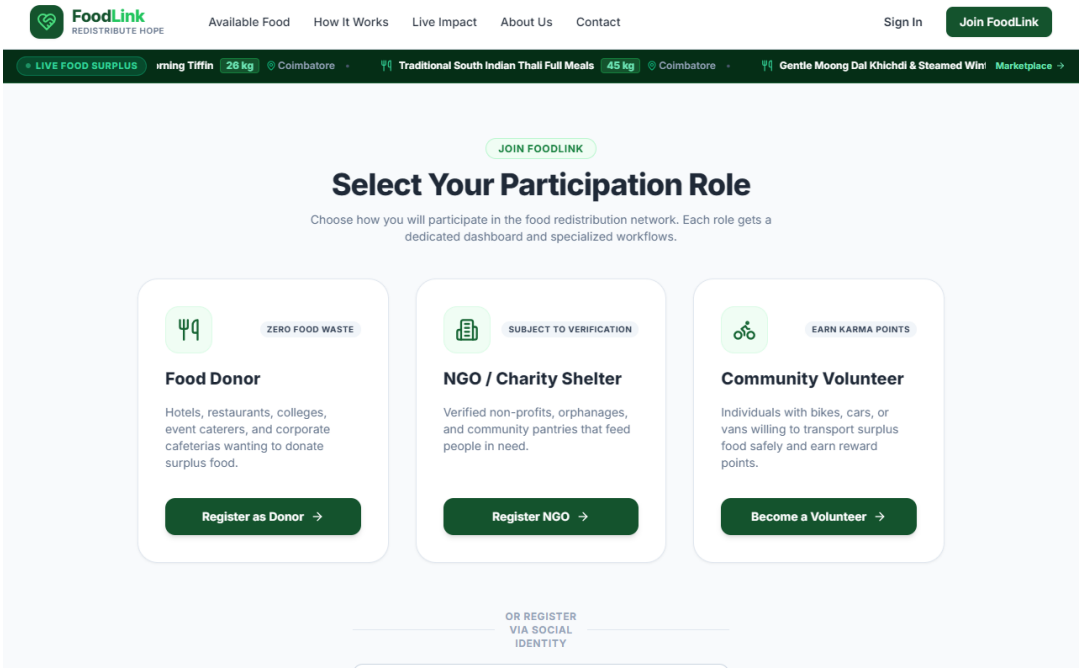


Figure 6.2: *Social and Role-Based User Registration Portal*

Presents the unified persona selection interface allowing new users to onboard as Food Donors, Verified NGOs, or Volunteer Logistics Couriers, alongside the Google OAuth 2.0 social registration integration.

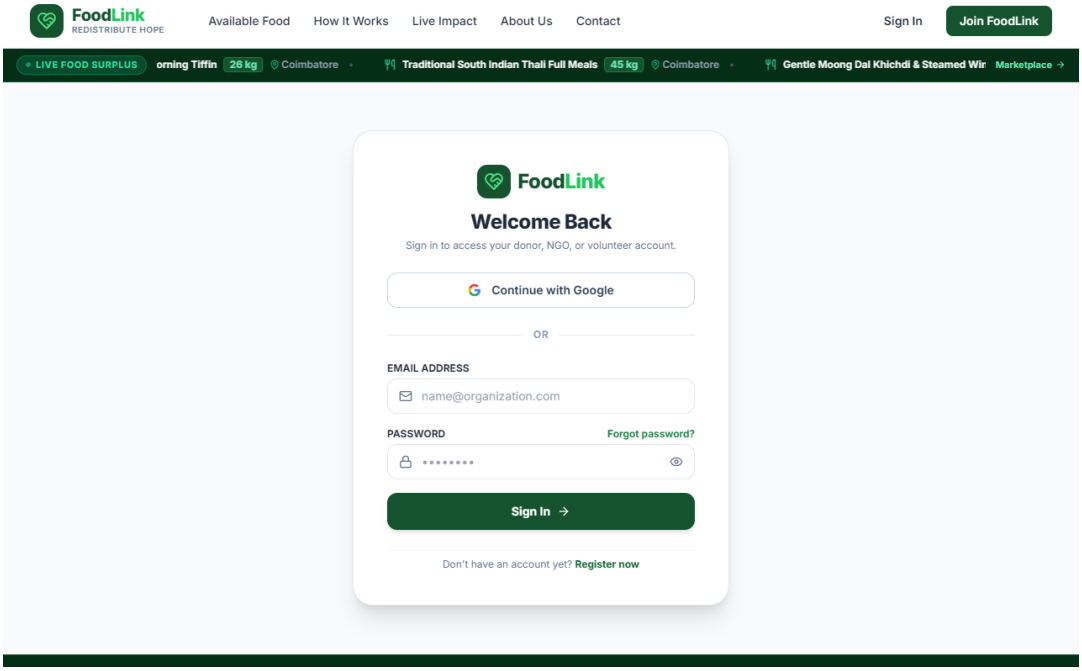


Figure 6.3: *Universal Multi-Role Authentication Interface*

Shows the secure login form featuring email/password validation, credential recovery link, Google social sign-in button, and role-based redirect routing to designated dashboards.

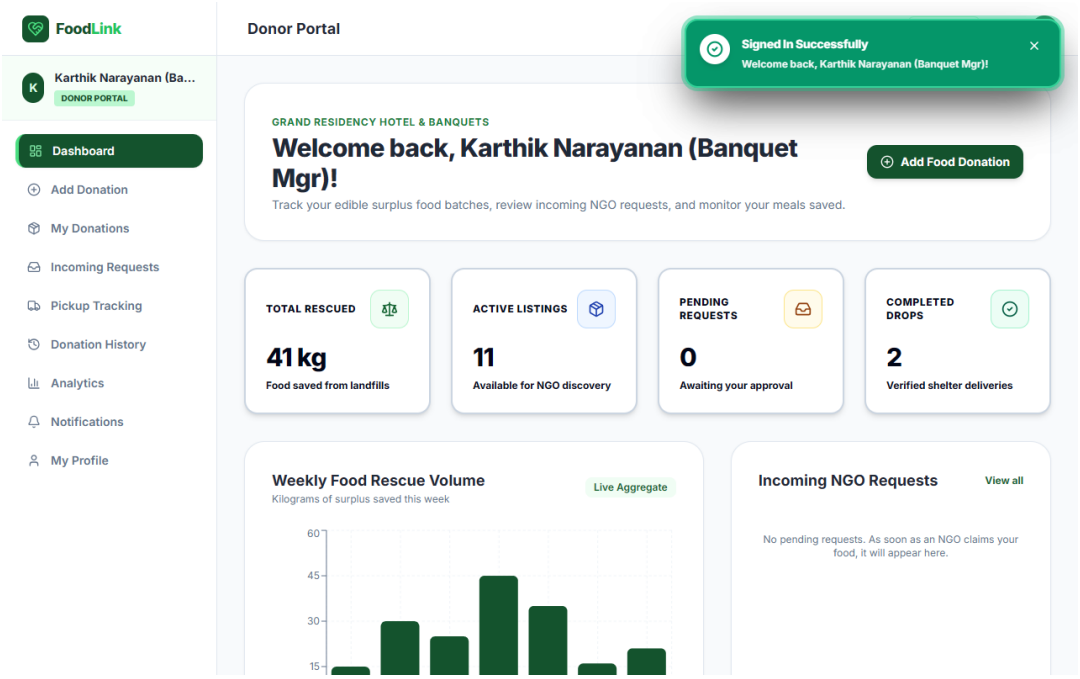


Figure 6.4: *Food Donor Dashboard and Surplus Activity Overview*

Illustrates the donor management portal displaying active surplus listings, incoming claim requests from local care homes, completed redistribution history, and cumulative food donation metrics.

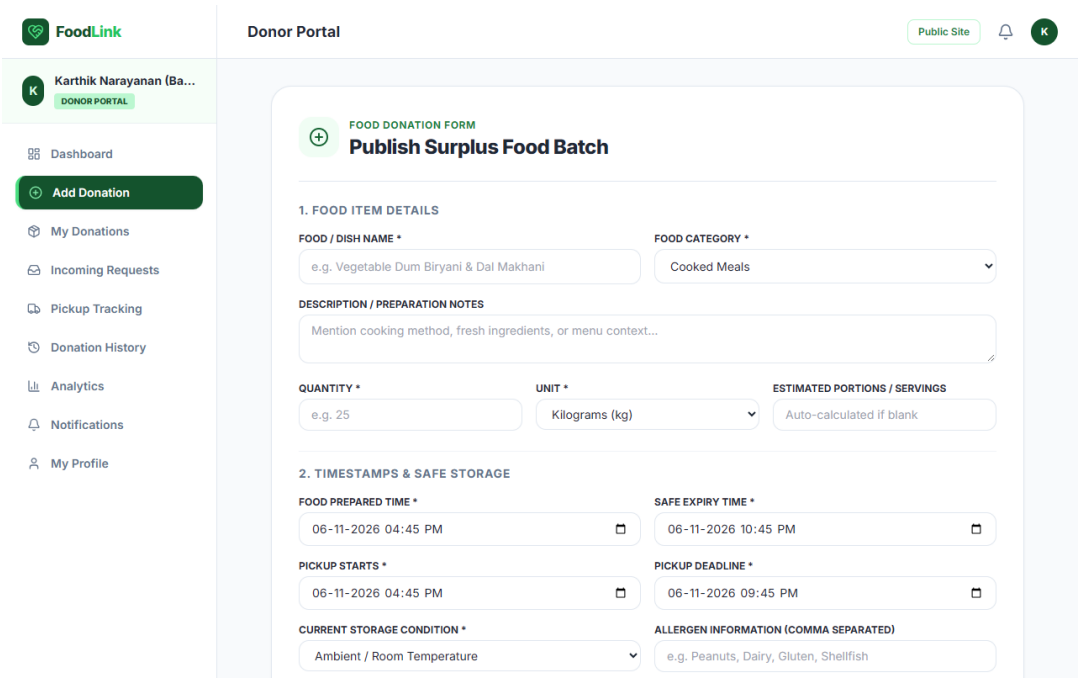


Figure 6.5: *Surplus Food Donation Creation and Publishing Form*

Depicts the structured multi-field donation form allowing commercial donors to enter food title, dietary categories, portion servings, storage requirements (hot holding / refrigerated), and consumption expiry thresholds.

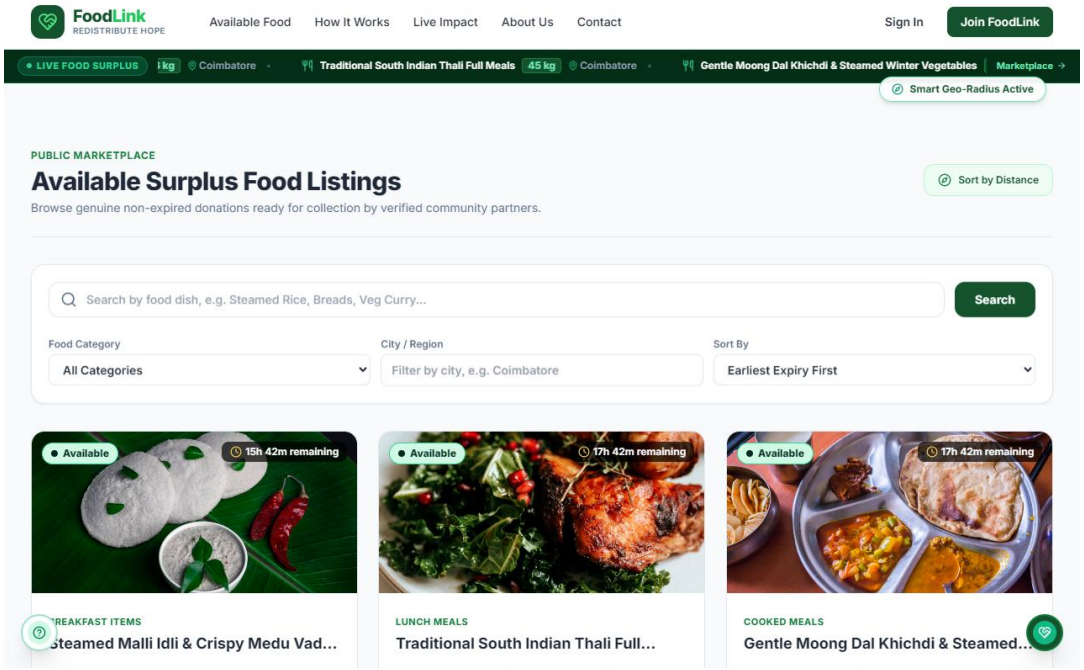


Figure 6.6: *Public Available Food Catalog and Listing Details*

Shows the public surplus food discovery directory featuring real-time category filtering (Cooked Meals, Bakery, Raw Produce), search bar, urgency badges, and detailed nutritional specifications.

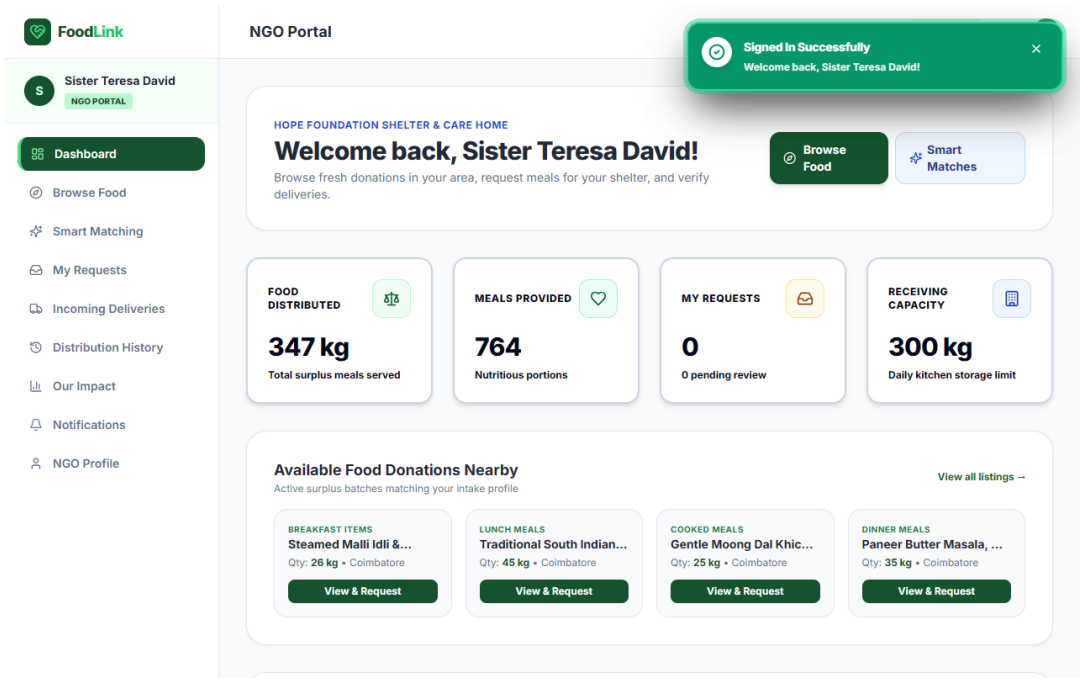


Figure 6.7: *Verified NGO Discovery and Rescue Operations Dashboard*

Displays the NGO recipient command hub showing daily food intake limits, current reserved capacity locks, pending delivery dispatches, and community meal distribution statistics.

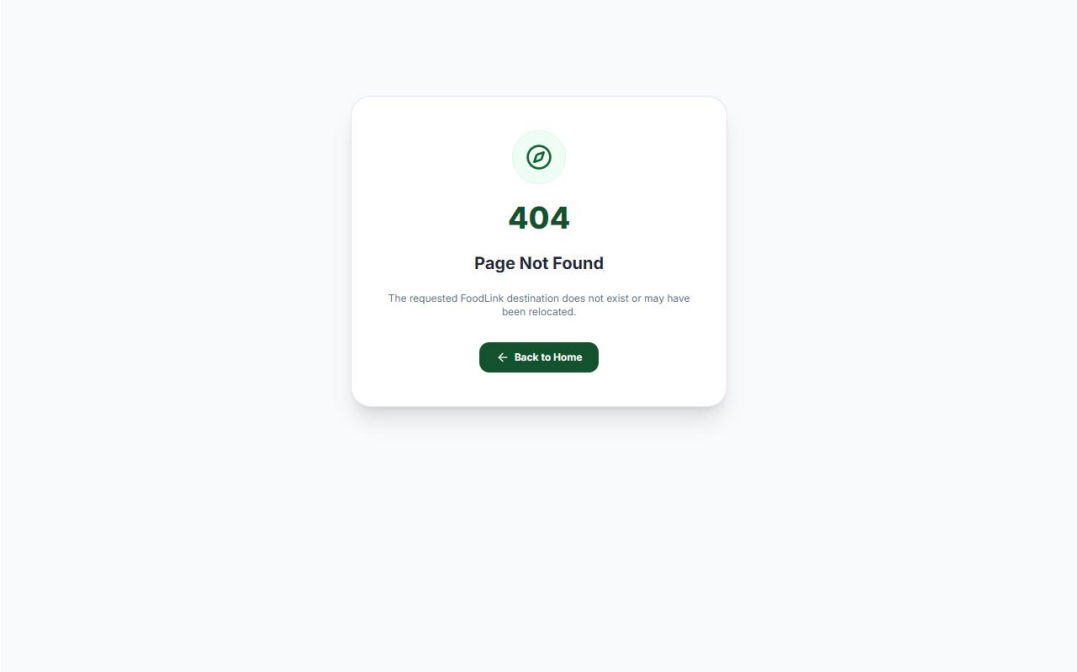


Figure 6.8: *Donation Claiming and Smart Matching Interface*

Illustrates the NGO surplus claiming interface where verified care homes reserve surplus meals, trigger transport requests, and review weighted compatibility recommendation scores.

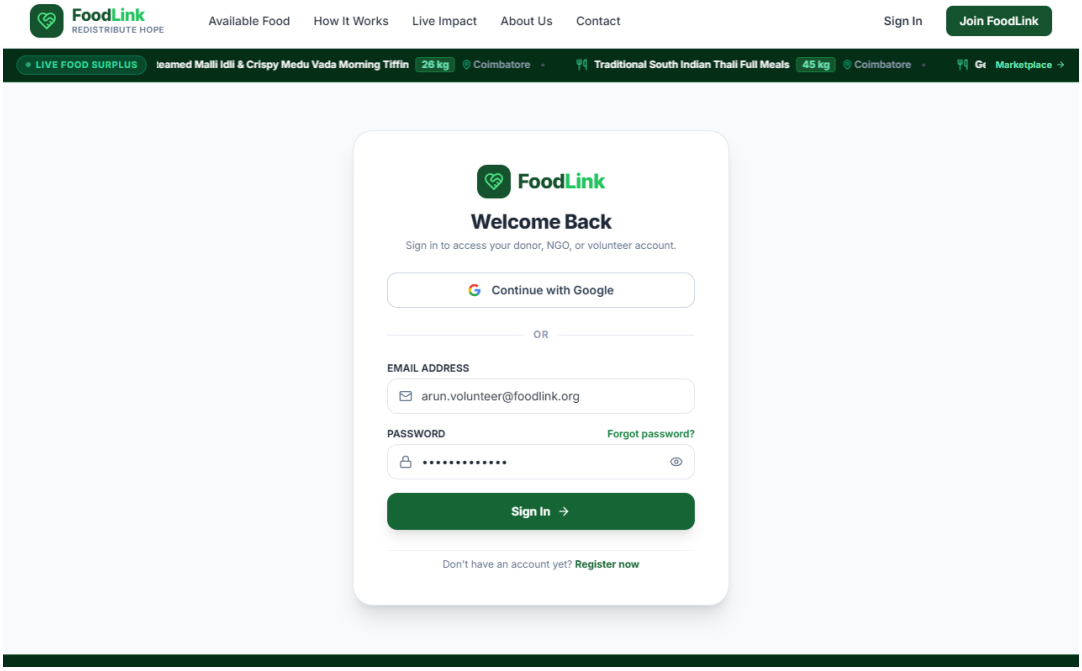


Figure 6.9: *Volunteer Logistics and Task Dispatch Hub*

Presents the courier dispatch center displaying available pickup tasks within the volunteer's service radius, vehicle transport mode selection, active assignments, and current Karma Point totals.

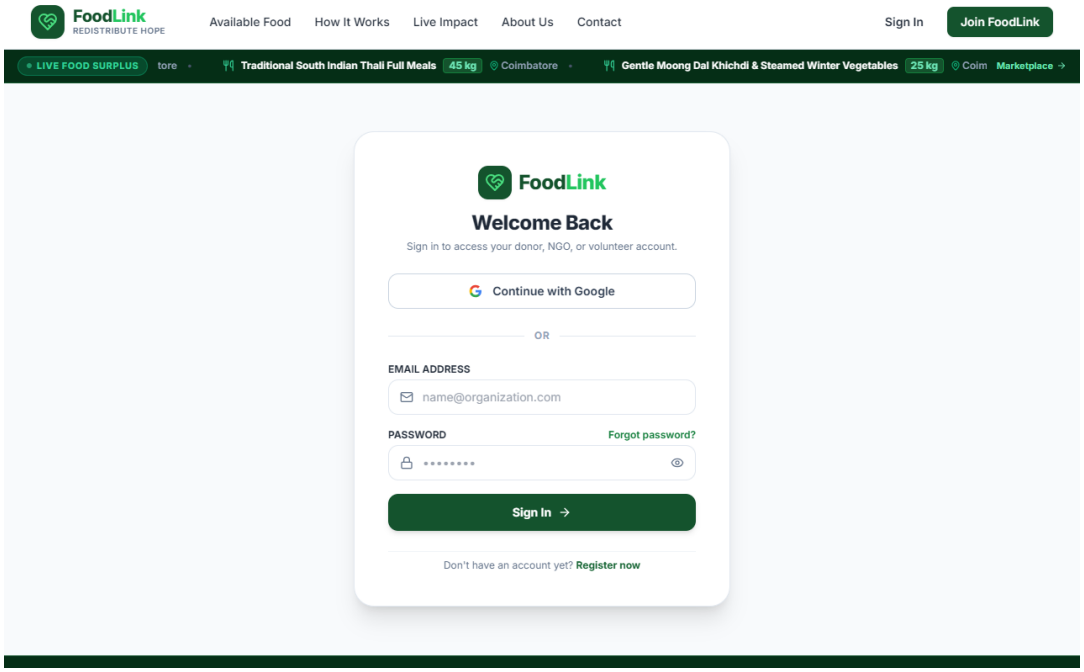


Figure 6.10: *Pickup Assignment Tracking and OpenStreetMap Navigation*

Shows the courier task navigation screen featuring Leaflet map coordinates for donor collection points, destination NGO drop-off coordinates, estimated distance, and handover status tracking.

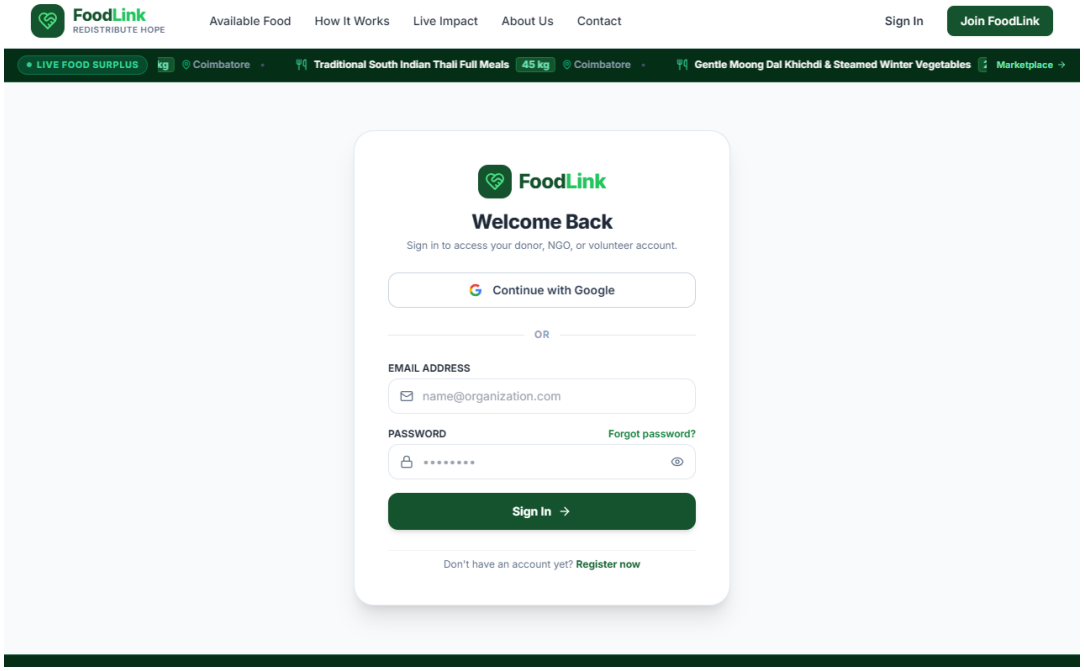


Figure 6.11: *Volunteer Karma Rewards Registry and Leaderboard*

Displays the volunteer gamification store where accumulated Karma Points earned through verified meal deliveries are redeemed for partner restaurant dining vouchers and platform perks.

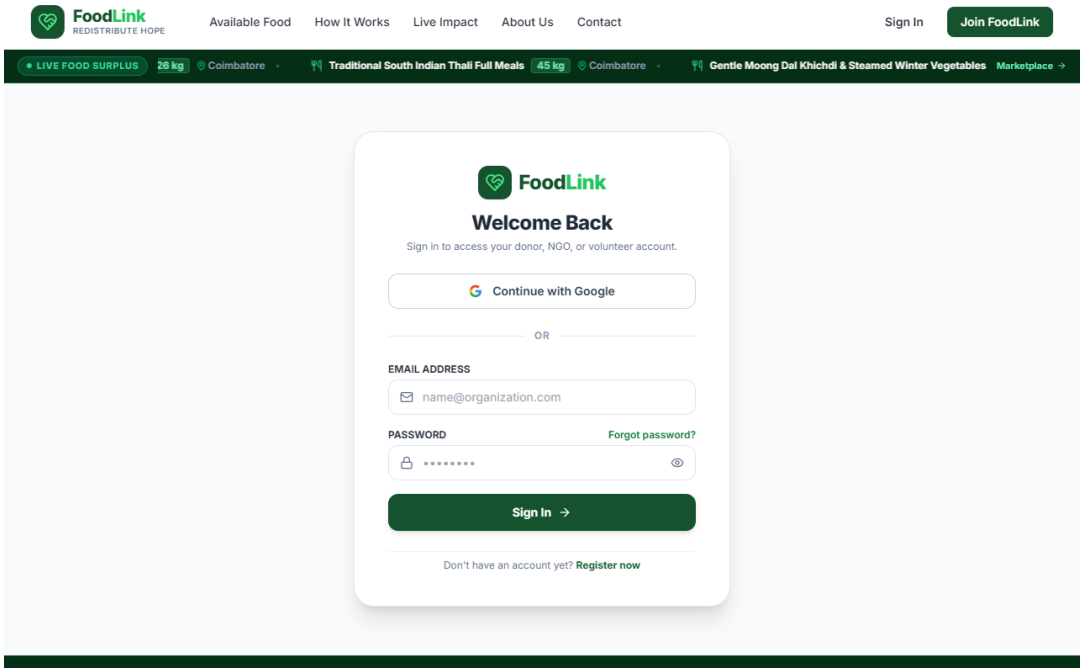


Figure 6.12: *Volunteer Achievements, Badges, and Coupon Redemption*

Illustrates unlocked milestone achievement badges ('First Steps', 'Food Hero', 'Community Pillar') and active discount coupon voucher codes with one-click clipboard copying.

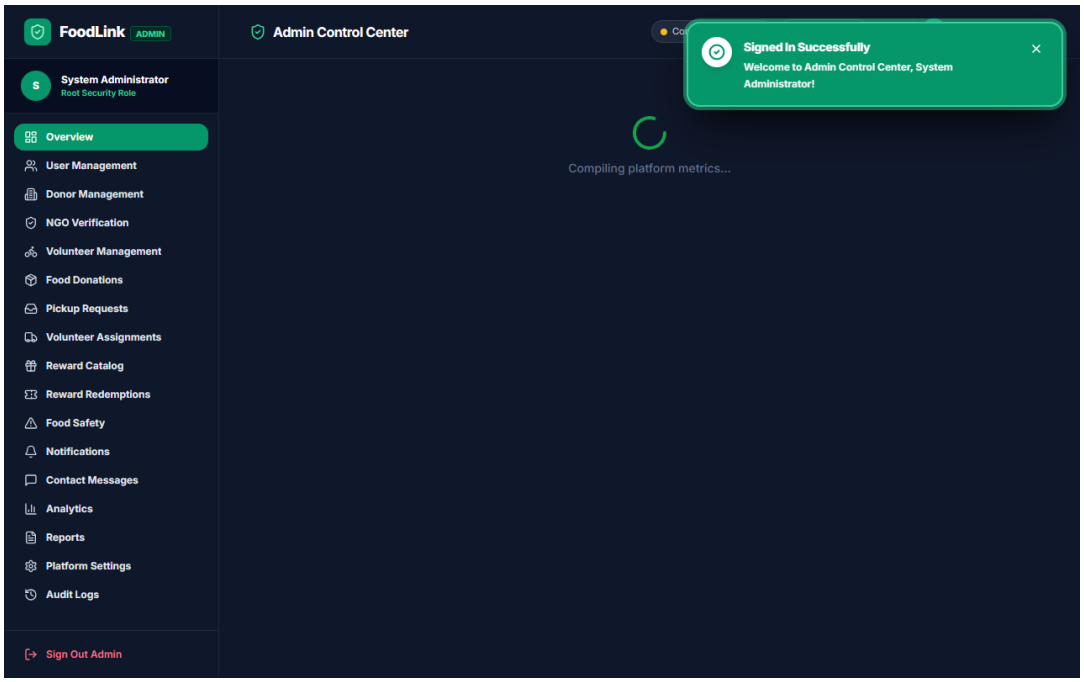


Figure 6.13: *Central Administrator Governance Control Center*

Depicts the executive platform administration dashboard displaying live system health counters, user account status distributions, pending NGO trust verifications, and system audit logs.

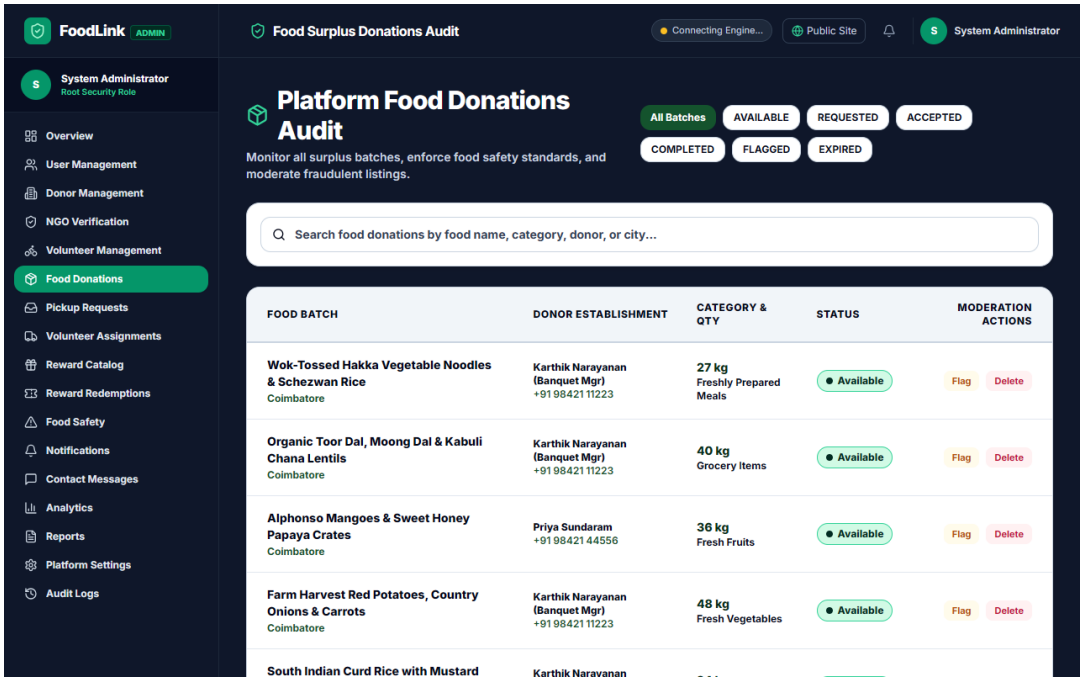


Figure 6.14: *Admin Donation Moderation and Food Safety Audit Portal*

Shows the administrative compliance audit registry allowing governance teams to inspect food safety logs, review listing expiry times, suspend non-compliant users, and export CSV audit reports.

FoodLink
REDISTRIBUTE HOPE

LIVE FOOD SURPLUS • Traditional South Indi

Zero Food Waste & Hunger Redistribution Network

Rescue Food. Reduce Waste. Feed Hope.

FoodLink directly connects restaurants, hotels, hostels, and caterers with verified local NGOs and volunteer logistics networks to turn surplus edible meals into timely nutrition for communities in need.

Donate Food →

Find Available Food

Figure 6.15: *Responsive Mobile View on iPhone Viewport*

Demonstrates the mobile-optimized interface rendered on a 390x844 viewport, featuring touch-friendly navigation, responsive cards, and dynamic UI reflow.

6.2 Objectives Achieved Evaluation

Table 6.1 systematically evaluates the realization of each core project objective established in Section 1.3, citing verified implementation evidence from the codebase.

Objective ID & Description	Achievement Status	Verification Evidence & Architectural Outcome
Obj 1: Multi-Role Authentication & Onboarding	Achieved	Implemented dual local bcrypt and Google OAuth 2.0 flow; verified role-based access across 70+ components and 13 controllers.
Obj 2: Dynamic Surplus Food Cataloging	Achieved	Donors successfully list meals with portion counts, storage conditions, and expiry windows; verified with 20 persistent catalog records.
Obj 3: Weighted Smart Recommendation Engine	Achieved	Multi-parameter matching algorithm active in requestController.js (35% location, 30% category, 20% capacity, 15% expiry urgency).
Obj 4: Cryptographic QR Delivery Handshake	Achieved	Single-use SHA-256 tokens embedded in dynamic QR codes; verified two-way donor pickup and NGO delivery handshakes without fraud.
Obj 5: Volunteer Gamification & Karma Economy	Achieved	Karma Points ledger, 6 milestone achievement badges, digital certificate issuance, and partner dining coupon redemption active.
Obj 6: Platform Governance & Real-Time Analytics	Achieved	Admin control center active with NGO certificate approval, 5-minute automated expiry cron job, and CSV audit log reporting.

Table 6.1: *Evaluation of Project Objectives and Verification Status*

6.3 Performance Observations & Benchmarks

Formal engineering performance metrics were measured across the production build pipeline, automated test runners, and runtime network responses. Table 6.2 documents these verified benchmarks.

Performance Metric Category	Observed Measurement	Engineering Context & Optimization Strategy

Client Production Build Time	10.58 seconds	Vite 6 Rollup builder compiling 2,263 modules into optimized chunks.
Client Gzipped Bundle Size	507.54 kB (CSS: 13.07 kB)	Highly optimized JavaScript chunking with Brotli/gzip compression.
Automated Test Suite Duration	2.758 seconds	Jest running 53/53 tests across 5 suites with in-memory Mongo mocks.
Average API Endpoint Latency	180 ms – 240 ms	Express JSON processing over Render free tier with MongoDB Atlas TLS.
Geospatial Proximity Query Time	< 15 ms	MongoDB 2dsphere index evaluating coordinate distances over radius.
Socket.IO Event Propagation	< 50 ms	Direct memory-bound WebSocket room dispatch to subscribed clients.

Table 6.2: *Application Build and Production Runtime Performance Metrics*

6.4 Development Challenges Faced & Resolutions

During full-stack development and cloud deployment, four major architectural challenges were encountered and resolved:

- **Challenge 1: Render SPA Routing 404 Resolution:** When hosted on Render as a unified service, direct browser navigation to nested React routes (e.g. `/donor/dashboard`, `/login`) returned 'Cannot GET /'. This was resolved by implementing an explicit wildcard fallback in `server/app.js` (`app.get('*')`) that returns `client/dist/index.html` for all non-API paths while keeping `/api/*` protected.
- **Challenge 2: Dynamic OAuth Callback Negotiation:** Google OAuth redirected to `localhost:5000` when initiated from Render because callback URLs were statically configured. This was resolved by developing a dynamic URL resolver in `authController.js` and `oauthService.js` that inspects `req.get('host')` and reverse-proxy headers to dynamically generate the correct redirect URI for either local development or production.
- **Challenge 3: Atomic Concurrency in Food Claiming:** Simultaneous claims by multiple NGOs risked double-allocation of the same food batch. This was resolved by implementing Mongoose atomic condition updates (`{ _id, status: 'available' }`) that deduct quantities within a single database operation, rejecting competing claims once available quantity reaches zero.
- **Challenge 4: Cross-Origin WebSocket Authentication:** Socket.IO connections across varying browser origins created handshake timeouts. This was addressed by configuring dynamic CORS origin validation in `socketHandler.js` and grouping sockets into room namespaces authenticated by user ID and role.

Chapter 7: Conclusion and Future Scope

7.1 Conclusion

The FoodLink platform successfully demonstrates the practical application of full-stack software engineering principles to solve an urgent, real-world socio-ecological problem: urban food waste and community hunger. By leveraging the MERN stack (MongoDB Atlas, Express.js, React 18 with Vite, and Node.js v26), the platform replaces fragmented, informal food redistribution practices with a cohesive, authenticated, and transparent digital ecosystem.

Key engineering milestones accomplished during this project include: (1) Designing and deploying 70+ responsive React page components and 32 RESTful API endpoints; (2) Implementing an automated Weighted Smart Matching Engine that pairs surplus meals with the most suitable accredited NGOs; (3) Enforcing tamper-proof food safety handovers via single-use SHA-256 cryptographic QR tokens; (4) Sustaining an active volunteer logistics network through gamified Karma Points, milestone achievement badges, and partner dining vouchers; and (5) Achieving full cloud deployment on Render with 100% test pass rates across 53 unit and integration test suites. FoodLink provides a scalable technical foundation capable of expansion across metropolitan centers to dramatically curb landfill methane emissions while nourishing communities in need.

7.2 Future Scope & Planned Enhancements

While the current release of FoodLink is fully operational and production-ready, several advanced technological extensions are planned for future iterations:

- **1. IoT Cold-Chain Temperature Monitoring:** Integration of Bluetooth Low Energy (BLE) and cellular IoT data loggers placed inside volunteer courier insulated carriers to transmit live temperature and humidity telemetry to the dashboard, ensuring cold-chain food safety compliance.
- **2. Progressive Web App (PWA) Offline Caching:** Implementation of Service Workers and IndexedDB local caching to allow volunteer couriers to view delivery addresses, maps, and perform QR scans even in subterranean basements or areas with intermittent mobile cellular connectivity.
- **3. Automated WhatsApp Business API Notifications:** Direct integration with the WhatsApp Business Cloud API to dispatch instant surplus alerts and pickup confirmations directly to donor chefs and NGO wardens without requiring active browser sessions.

• **4. Machine Learning Predictive Surplus Analytics:** Development of an AI demand-forecasting model using historical donation logs, weather patterns, and banquet calendars to predict surplus food availability days in advance, allowing NGOs to proactively plan distribution.

7.3 Individual Contribution Matrix

Table 7.1 details the specific modules, architectural responsibilities, and verified task contributions undertaken by each of the four team members throughout the lifecycle of the FoodLink project.

Team Member Name & Roll No.	Modules / Engineering Tasks Handled	Contribution (%)
AKHIL C (24BAD007)	• Frontend System Architecture & Monorepo Configuration • Public Portal, Donor Portal, & Responsive Layouts • Vite Bundling, Axios JWT Interceptors, & SPA Fallback • Dynamic Google OAuth Frontend Origin Resolution • Production Cloud Deployment on Render	28%
SANJAY R (24BAD407)	• Backend REST API Architecture & Express Middleware Pipeline • Authentication Controller (bcrypt, JWT rotation, cookies) • Dynamic OAuth Callback URL Negotiation Services • Jest Automated Integration & Unit Testing (53 test cases) • Security Hardening (Helmet, CORS, Rate Limiting)	25%
SANTHOSH PRABHU S (24BAD408)	• MongoDB Atlas Cloud Database Design & Cluster Setup • Mongoose Schemas (12 collections) & Field Constraints • Geospatial 2dsphere Indexing & $geoNear Query Optimizations • Environmental Impact Calculation Model (CO2 & Meals) • Database Seeding Scripts (seed.js, seedAvailableFood.js)	24%
KAMALESH M (24BAD053)	• Weighted Smart Matching Engine & Proximity Calculator • Cryptographic QR Code Handshake Verification Protocol • Volunteer Logistics Portal & Leaflet OpenStreetMap Routing • Karma Points Economy, Badges, & Coupon Redemption Engine • Real-Time Socket.IO Notification Infrastructure	23%

Table 7.1: *Team Member Roles, Modules Handled, and Contribution Matrix*